Predicting flow with Back Propagation Neural Networks

F328799

Contents

Chapter 0

Outline

The aim of this project is to predict the flow-rate of the River Ouse at Skelton in advance to predict the flood potential. We will use a Back Propagation Neural Network to predict the flow-rate of the river, using data from the surrounding catchment area. This data includes river flow-rate from Skip Bridge, Crakehill and Westwick, as well as rainfall data from East Cowton, Arkengarthdale, Snaizeholme and Malham Tarn. The data was collected between 01/01/1993 to 31/12/1996.

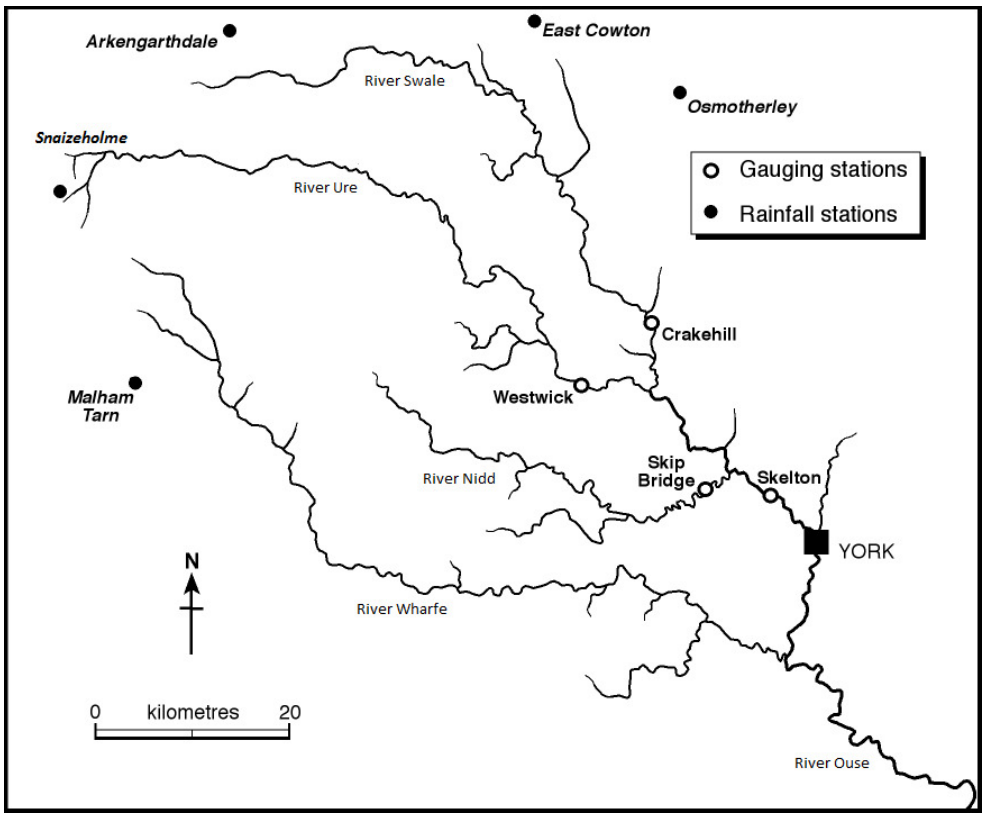


Figure 0.0.1: River Ouse Catchment

The sections of this report will include:

1. Data Pre-processing
2. Implementing the Neural Network
3. Model Training
4. Evaluation

0.1 System Environments

The ANN will be programmed using python 3.13.2 with a conda virtual environment. The libraries used will include:

- numpy
- pandas, sqlalchemy

- openpyxl
- sqlite3
- matplotlib
- seaborn
- time
- matplotlib's PdfPages

The data will be stored in an SQLite database, this is for ease of access and to keep the data in a single location, with concise SQL commands to access the data.

Chapter 1

Data Pre-processing

1.1 Importing data

The first step is to import the data from the give xlsx file into the database. This will be done using the pandas library to read the data from the xlsx file and then using the sqlalchemy library to write the data to the database. For simplicity, I created a second xlsx file without the first row of headers, this is so that the data can be read into the database without any issues.

This code shows how I handled the non-numerical data in the xlsx file. It is then imported into the database using the sqlalchemy library. Line 16 is used to remove non-numerical data when importing into the database. This is to avoid any type-errors when the create_engine creates the dataBase.

```
1  excel_import.py
2
3      import pandas as pd
4      import numpy as np
5      import sqlite3
6      from sqlalchemy import create_engine
7
8
9      def data_import(dataBase:pd.DataFrame, dataTable:str, excelFile:str):
10         '''Function for importing Excel data into a database'''
11
12         # Read the excel file and store the data in a DataFrame
13         excelFile = pd.read_excel(excelFile)
14
15         # Check for non-numeric values in non-first columns and replace them with na points
16         excelFile.iloc[:, 1:] = excelFile.iloc[:, 1:].apply(pd.to_numeric, errors='coerce').fillna(np.nan)
17
18         # Create a connection to the SQLite database (or create it if it doesn't exist)
19         conn = sqlite3.connect(dataBase)
20         cursor = conn.cursor()
21
22         # Drop the table if it exists so the database can be recreated from scratch for testing
23         cursor.execute(f'DROP TABLE IF EXISTS {dataTable}')
24
25         # Use SQLAlchemy to import the DataFrame into the SQL database
26         engine = create_engine(f'sqlite:///{dataBase}')
27         excelFile.to_sql(dataTable, engine, if_exists='replace', index=False)
28
29         # Commit the changes and close the connection
30         conn.commit()
31         conn.close()
32         print('Data imported successfully!')
```

All code will be enclosed in the additional files included

1.2 Extreme Outliers

Using the database as a source of data, we can now begin to pre-process the data. Using matplotlib, we can plot the data to see if there are any trends or patterns in the data, along with any outliers that may need to be removed.

In the given dataset, there were a number of values that were not possible, such as negative flow rates and rainfall values in the thousands. These values need to be removed from the dataset to ensure the model is accurate. To remove these and other 'extreme' outliers, I used the inter-quartile range (IQR) method. When I implemented this, I used a variable to multiply by the IQR to get the upper and lower bounds, allowing me to refine the data to a point I thought was acceptable.

A typical weight for extreme values is 3, this is because the IQR is the range between the 25th and 75th percentile, so multiplying by 3 gives a range of 99.7% of the data (when its normally distributed). However, as the data is skewed heavily towards 0, the 75th percentile still encapsulates a large amount of possible data within a wide range. After experimenting with different values, the results were as follows:

Weight	Amount Removed
1.5	2031
3.0	782
6.0	176
10.0	50

Table 1.1: Results of different weights for IQR

From testing, I decided that 10 was the best weight to use, as it removed the most extreme values without removing too much data. The remaining data looks real and is within a reasonable range for realistic rainfall and flow rates.

1.3 Averaging and Deviations

The data has had its extreme outliers removed, but it is still not in a usable state. The data needs to be averaged and standardised to ensure that the data is in a usable state for the neural network.

> *For context: These functions are called within an iterative loop that acts on them for each of the columns in the database. columnData is a pandas data frame of the date with that dates record.*
>
> *The data points are uniformly iterated with columnData.itertuples(), hence the use of the point[n] convention to access the data.*
>
> *When a data point is removed, it is stored in a dictionary object, with the key being the column name and the value being a dictionary of the removed data points. All code will be enclosed in the additional files included.*

Our IQR with extreme weighting method does not account for large amount of rainfall in a dry spell, or low flow rate at a time of flooding. To account for this, we will use methods to determine if the data is within a reasonable range, over a period of time.

Rainfall and especially flow rate data move in trends, so to best capture the data we will use a condensed version of the data, with only its neighbors. We will calculate our outliers over this as rainfall in summer will not be as high as in winter (it's still England though), so calculating the averages of the whole data set will not be accurate.

data_cleaning.py

```
import numpy as np
import pandas as pd

def local_range_df(columnData:pd.DataFrame, dataPoint:tuple, columnLength:int, windowSize:int) -> pd.DataFrame:
    '''Function to create a smaller range of data around a point'''
    windowSplit = round(windowSize/2)

    # Defines the start and end of the range
    if int(dataPoint[0])-windowSplit < 0:
        '''If the point is close to 0'''
        start = 0
        end = windowSize-int(dataPoint[0])
    elif int(dataPoint[0])+windowSplit > columnLength:
        '''If the point is close to the end of the DF'''
        end = columnLength
        start = columnLength - windowSize
    else:
        '''If the point is in the middle of the DF'''
        start = int(dataPoint[0])-windowSplit
        end = int(dataPoint[0])+windowSplit

    # Create a range of the data around the point
    localData = columnData.loc[start:end]

    return localData
```

We will pass this local data to the standard deviation and IQR functions to calculate the upper and lower bounds of the data.

Flow rate of a river rises and falls in patterns, so to best catch outliers we will use a mean and standard deviation over a period of time.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def standard_deviation_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the standard deviation of a range of data'''

        # Calculate the mean and standard deviation of the DataFrame
        deviation = columnData[columnName].std(axis=0, skipna=True, numeric_only=True, ddof=1)
        average = columnData[columnName].mean(axis=0, skipna=True, numeric_only=True)

        # Calculate the upper and lower bounds
        lowerBound = average - (deviationWeight * deviation)
        upperBound = average + (deviationWeight * deviation)

        return lowerBound, upperBound
```

This method determines if the data follows the trend of its neighbors, and if it does not, it is removed from the dataset. One day of high flow rate isn't a realistic measurement as the river constantly flows.

However, rainfall can be more sporadic, so we will once again use a weighted IQR method to remove extreme values.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def iqr_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the IQR of a range of data'''

        # Calculate the IQR of the range
        Q1 = columnData[columnName].quantile(0.25)
        Q3 = columnData[columnName].quantile(0.75)
        IQR = Q3 - Q1

        # Calculate the upper and lower bounds, round to 3 decimal places
        lowerBound = round(Q1 - (deviationWeight * IQR), 3)
        upperBound = round(Q3 + (deviationWeight * IQR), 3)

        return lowerBound, upperBound
```

This will be done over a period of time, as a single day of high rainfall is possible in a storm, before a low rainfall spell occurs again. By splitting the data this way we allow the more accurate flow rate data to be used in the model, while still allowing for the sporadic nature of rainfall.

Once the data points have been removed from the dataset, we can replace them with the average of the surrounding data points. This is to ensure that the data is continuous and that the model can be trained on the data.

1.4 Interpolation

When the algorithm removes the data points, it leaves gaps in the data. I made this decision to separate the extraction and interpolation for two reasons:

1. To allow for efficiency of interpolation
2. For accuracy, as im only using the data points that were recorded, not interpolated

To interpolate the data, I used a simple moving average (SMA) method to fill the gaps. I implemented this with a variable window size for the SMA to allow for flexibility in the data. The numpy array allows for skipping of NaN values, so the data can be interpolated in places where I have removed the data points. However, too small of a window size ends up with a variation of a forward fill at points with data removed. I decided that a window size of 6 was the best for the data, allowing for trends to show without ending up with a forward fill.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def simple_moving_average(columnData:pd.DataFrame, columnName:str, windowSize:int, removedDict:dict, updatedDict:dict) -> tuple[dict, dict]:
        '''Function to update empty points in the dataset'''

```

```
        # Find all NaN values in the column
        emptyPoints = columnData[columnData[columnName].isna()]

        # Iterate through the empty points
        for point in emptyPoints.itertuples():
            '''Iterate through the empty points in the DataFrame'''

            # Get the local range of data around the point
            localData = local_range_df(columnData, point, len(columnData), windowSize)

            # Calculate the average of the range
            average = round(localData[columnName].mean(skipna=True),)

            # Update the statistics
            removedDict[columnName][point[1]].append(float(average))

            # For visualisation purposes, store some of the updated data in separate dictionary
            try:
                updatedDict[columnName][point[1]].append(float(average))
            except:
                None

            # Replace the empty point with the average
            columnData.loc[point[0], columnName] = average

        return removedDict, updatedDict
```

The outputs of this function are two dictionary objects. As explained in the context note, the dictionary is used to store the data points that were removed from the dataset. Both of them are the same, except updatedDict is used for visualisation purposes. As the extreme values are present in the removedDict, when plotted they skew the graph so you can't see the trends in the data.

I then go on to use the updatedDict to plot the data, showing the trends in the data, along with the removed data and the ones that replaced it. This is to show the accuracy of the interpolation and to show the trends in the data. The removedDict is used to update the SQL database, so the data can be used in the neural network; we need a record of data that was changed to be able to update it. If we instead saved it to the data frame, we would lose the location of updated data points. This would result in updating the whole database with the whole data frame, wasting a lot of time and resources replacing points with values they already have. The database has 11,688 data points, with 992 (for variables stated in figure 1.5.2) removed and replaced. That's 8.5% of the data and more than 10 times more efficient only updating those.

1.5 Plotting trends

Once the data has been cleaned and interpolated, we can plot the data to see the trends in the data. We can also make judgments on the variables that determine which point are outliers and which are not.

Lets use the data at Crakehill as an example location for visualisation:

Figure 1.5.1 show us the data at Crakehill, with the unrealistic data present. As I stated before, the data scale is affected by the outliers, so the trends in the data are not visible.

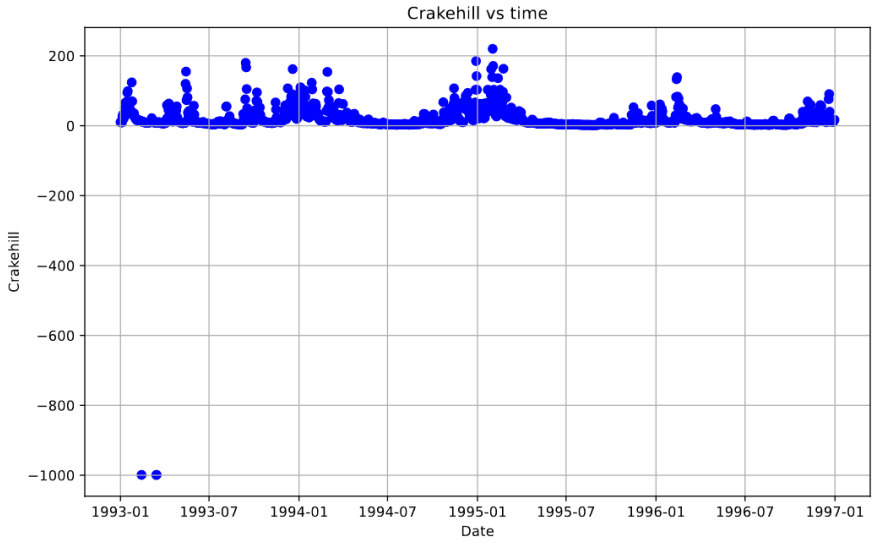


Figure 1.5.1: Crakehill Data with unrealistic data

Figure 1.5.2 shows the data at Crakehill, with the unrealistic data points removed, but still containing outliers

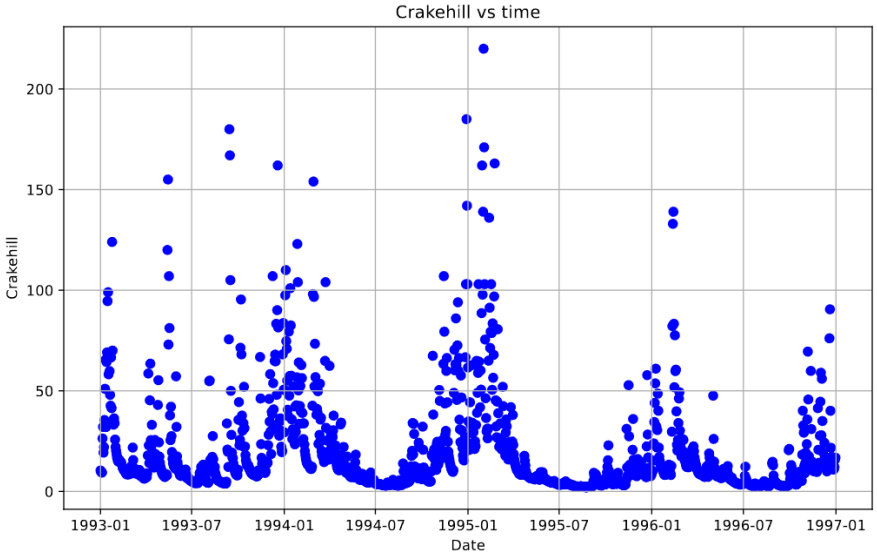


Figure 1.5.2: Crakehill Data without unrealistic data

Finally, figure 1.5.3 shows us the result of our interpolation, with the data points that were removed and replaced. Blue points are the data we are using, red points are the data that was removed and green being what replaced them.

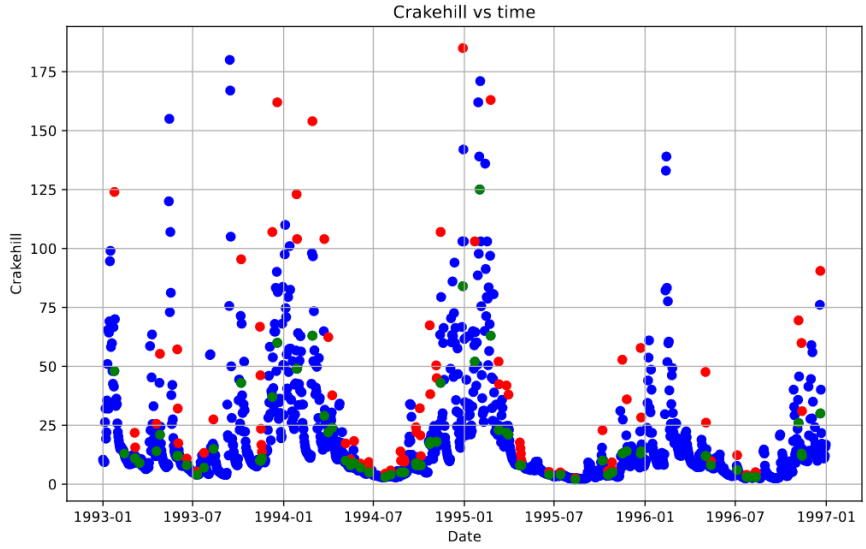


Figure 1.5.3: Crakehill Data interpolated

This specific figure has:

- IQR weight of 1.5
- Standard deviation weight of 2
- Window size of 10 for removal
- Window size of 6 for interpolation
- 91 data points removed and replaced (992 across all locations)

When looking at figure 1.5.2, we can see that in 1993, the flow rate data was much more sporadic than the following years, then in figure 1.5.3, the data is clearly less sporadic and follows the trends more closely. This data should now be considered usable to train our Multi-Layer Perceptron (MLP) model. The database can now be updated with the new data.

1.6 Correlation

To best predict our data, we need to choose the correct inputs that will give us the best results. We need to aim to a high correlation of the data points to the predictand, while also avoiding correlation between predictors.

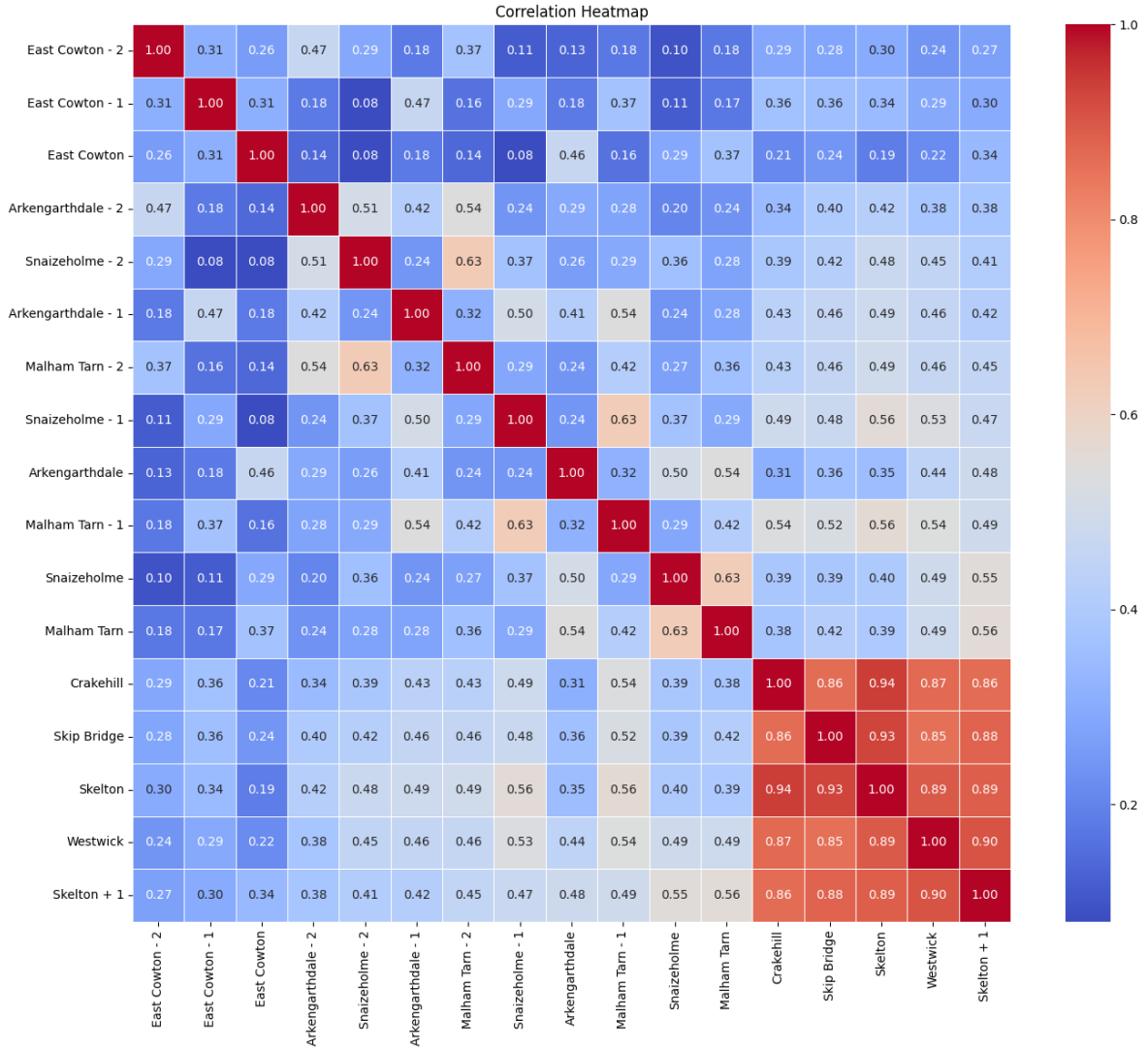


Figure 1.6.1: Correlation between the data points

FIgure 1.6.1 shows the correlation between the data points, which we can use to determine the best predictors for our model. As we are trying to to predict the flow rate at Skelton for the day after a data point, the correlation map has been ordered by the correlation to Skelton + 1. The highest correlation points are the other flow rate data points, with the highest being the flow rate at Westwick. They are all highly correlated to Skelton +1, but also to each other. This is to be expected as they are all flow rate data points of the same river. An interesting point to note is that Westwick is the least correlated to Skelton on the day of measurement, but most correlated to the day after.

The rainfall data is much less correlated to the flow rate data, with the highest correlation being 0.56 at Malham Tarn. This is to be expected as the rainfall data is not directly related to the flow rate data, but the flow rate data is related to the rainfall data. The next highest correlated rainfall location is Snaizeholme (0.55), but this also has a high correlation to Malham Tarn. Refering back to figure 0.0.1, we can see that they are very close, and therefore the data will be very similar.
The next best correlations are at Malham Tarn -1 (0.49) and Arkengarthdale(0.48), but these are are not nearly as correlated each other and the previous two.

My initial choice for predictors will be Westwick, Snaizeholme and Malham Tarn-1, as they are the most correlated to Skelton +1, but also the least correlated to each other. Another option for predictors would be any of the other flow rate data points, and a combination of the 4 mentioned rainfall data points.

We can also use some more complex predictors to see if they are more correlated to the predictand. These complex predictors will be using averages of the data points, and adding select ones together to get a higher correlation.

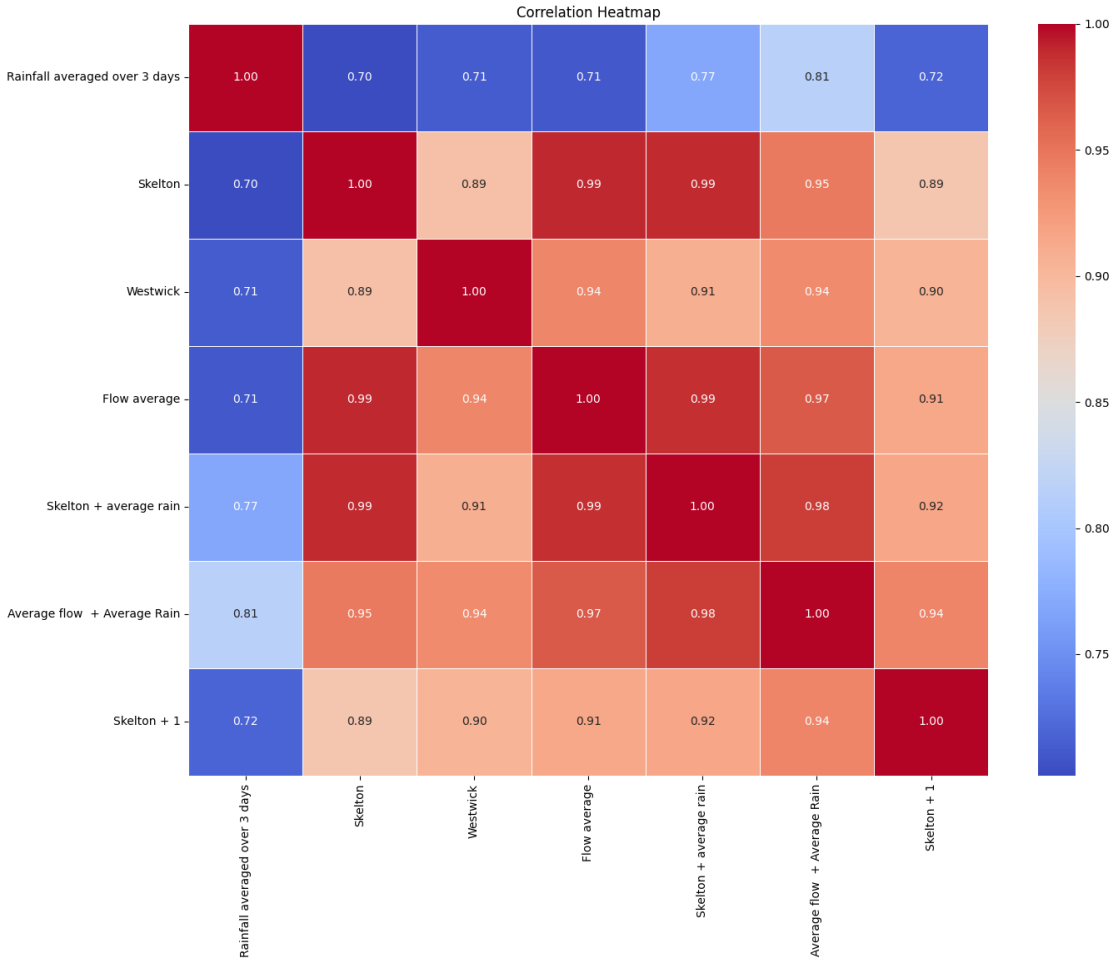


Figure 1.6.2: Complex Correlation between the data points

Figure 1.6.2 shows some of the selected complex data points. In a network of this size, the correlation between points is not as important as the correlation to the predictand, however I don't want to derive complex points that are too mathematically similar to one another.

correlation_mapping.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import reading_and_writing as rw

# Read the data
dataFrame = rw.read_data_all(dataBase, dataTable).iloc[:, 1:]

# Formulate the complex predictors
correlationTable['Skelton + 1'] = dataFrame['Skelton'].shift(-1)
correlationTable['Average flow  + Average Rain'] = (dataFrame['Flow average'] + dataFrame['Rainfall average'])
correlationTable['Skelton + average rain'] = (dataFrame['Skelton'] + dataFrame['Rainfall average'])
correlationTable['Flow average'] = (dataFrame['Skelton'] + dataFrame['Westwick'] + dataFrame['Skip Bridge'] + dataFrame['Crakehill']) / 4
correlationTable['Westwick'] = (dataFrame['Westwick'])
correlationTable['Skelton'] = dataFrame['Skelton']
correlationTable['Rainfall averaged over 3 days'] = dataFrame['Rainfall average'] + dataFrame['Rainfall average -1'] + dataFrame['Rainfall average -2']

# Drop na values
correlationTable = correlationTable.dropna()

# Calculate the correlation matrix
correlationMatrix = correlationTable.corr()

# Sort the correlation matrix
correlationMatrix = correlationMatrix.sort_values(by='Skelton + 1', axis=0, ascending=True)
correlationMatrix = correlationMatrix.sort_values(by='Skelton + 1', axis=1, ascending=True)


# Plot the correlation matrix
```

```
plt.figure(figsize=(16, 12))
sns.heatmap(correlationMatrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Chapter 2

Implementing the Neural Network

The neural network will be implemented using python and trained with the back propagation algorithm. The network will be a multi-layer perceptron (MLP), with at least one hidden layer.

Back propagation works by calculating the expected output of a network, then comparing it to the actual output; this is known as the forward pass. The algorithm then works backwards through the network, adjusting the weights of the network to reduce the error between the expected and actual output - the backward pass. This process is repeated for a given amount of epochs to help reduce the error to a minimal amount. This section is about the implementation of the algorithm and its improvements.

2.1 Data Splitting

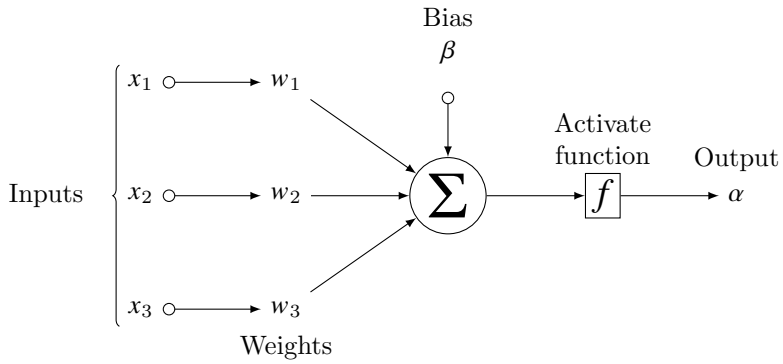
The data will be split into training, testing and evaluation data. To avoid seasonality in the data, we can use 2 years out of the 4 years for training, and the remaining 2 years for testing and evaluation(1 each). On paper this seems like a good idea, as each section has the same seasons for the same amount of time, in the same positions.

However, the data across the years is not equal, with 1993 having much more sporadic flow data than the following years. Additionally, there is the largest spike in flow data in the winter of 1994. This creates an issue where our training may not have sufficient data to predict the spike in flow rate into January 1995. We can try a random split of the data to see if this improves the model, but the results will only be known after the model has been trained.

2.2 Model Architecture

The MLP Architecture consists of repeated layers of nodes, each representing a 'neuron' in the network. Each node takes in x amounts of inputs, and eventually outputs a single value α.
To achieve this, each input is multiplied by a weight, and then summed together with a bias term. This bias term is introduced to avoid the network being stuck at 0, as the weights are randomly initialised, and input data can be 0. Once this sum has been calculated, it is passed through an activation function to give the output of the node.

This output then becomes the input for the next layer of nodes, repeating the process until the output layer is reached.



An example node in a neural network

2.3 Activation Functions

The activation function is a non-linear function that is applied to the output of a node.

2.3.1 Sigmoid

Sigmoid is a commonly used activation function, as it is simple and smooth. It allows for the network to learn complex patterns in the data, as it is non-linear. The function is as follows:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2.3.1}$$

The sigmoid function however can suffer from the vanishing gradient problem, where the gradient of the function becomes very small as the input becomes very large. This can lead to the network not learning as well, as the weights are not updated as much.

2.3.2 Tanh

The tanh function is similar to the sigmoid function, but is centred around 0, with a range of -1 to 1. The function is as follows:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{2.3.2}$$

The tanh function also suffers from vanishing gradients, as it has a similar structure to sigmoid, however its range is from -1 to 1, so it is more useful for normalising the data.

2.3.3 ReLU

The ReLU (Rectified Linear Unit) function is a simple function that is used in many neural networks. It is a piecewise function, with the function being 0 for all negative values, and the input for all positive values. The function is as follows:

$$f(x) = \max(0, x) \tag{2.3.3}$$

The ReLU function is used as it is simple and computationally efficient, and it does not suffer from the vanishing gradient problem. However, it can suffer from the dying ReLU problem, where the gradient of the function is 0 for all negative values, and the weights are not updated.

2.4 Cost Functions

There are 2 main loss functions that can be used in a neural network, the mean squared error (MSE) and the mean absolute error (MAE). The MSE is the most common loss function used in neural networks, as it is differentiable and convex. The MAE is less common, but is more robust to outliers in the data.

We use the cost function to calculated the error of the network, and then use the gradient of the cost function to adjust the weights of the network.

2.4.1 Mean Squared Error

The MSE equation is as follows:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(E_m - O_m)^2 \tag{2.4.1}$$

Where E_m is the expected output and O_m is the actual output.

We differentiate this with respect to output O_m to get the gradient of the cost function.

$$\frac{\partial \text{MSE}}{\partial O_m} = \frac{-2}{n}(E_m - O_m) \tag{2.4.2}$$

For large datasets, this can make the error of the network small and meaningless, leading to large training times.

2.4.2 Mean Absolute Error

The MAE equation is as follows:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|E_m - O_m| \tag{2.4.3}$$

We differentiate this with respect to output O_m to get the gradient of the cost function.

$$\frac{\partial \text{MAE}}{\partial O_m} = \frac{1}{n}\sum_{i=1}^{n}\frac{E_m - O_m}{|E_m - O_m|} \tag{2.4.4}$$

This gives us the following results for each case:

$$\frac{\partial MAE}{\partial O_m} = \begin{cases} \frac{1}{n}, & \text{if } O_m > E_m \\ -\frac{1}{n}, & \text{if } O_m < E_m \end{cases} \tag{2.4.5}$$

As you cannot divide by 0, there is not a smooth gradient at $O_m = E_m$.

Another potential issue is that as the gradient does not rely on the output, the network may not learn as well as with the MSE. This however is its strength, as it is more robust to outliers in the data, as it won't skew the data as much as the MSE.

For this project, I will use the MSE as the loss function, as the data has been cleaned and interpolated, and the outliers have been removed. However, I will use a variation of the MSE.

The function I will use is:

$$\frac{\delta MSE}{\delta O_m} = C(E_m, O_m) = E_m - O_m \tag{2.4.6}$$

This is because the gradient is dependent on the output, and the network will learn better with this function. It also removes the negative gradient as a negative gradient pushes the network in the wrong direction with my implementation of the back propagation algorithm. I have also removed the division by n as it will stifle my learning rate, as the data is already normalised and n is a constant.

2.5 Forward Pass

For n inputs, m hidden nodes in layer $\mathcal{L}$ and q output nodes.

The input matrix takes the n data points from the training data in the form:

$$\text{Matrix } \mathbf{I} \rightarrow [1 \times n] : \mathbf{I} = \begin{vmatrix} \mathbf{I}_1, & \mathbf{I}_2, & \cdots, & \mathbf{I}_n \end{vmatrix} \tag{2.5.1}$$

The weight matrix ω^1 is a $n \times m$ matrix, with each row representing the weights of a input nodes to each connected hidden node in the first hidden layer.

$$\text{Matrix } \omega^1 \rightarrow [n \times m] : \omega^1 = \begin{vmatrix} \omega^1_{11}, & \omega^1_{12}, & \cdots, & \omega^1_{1m} \\ \omega^1_{21}, & \ddots & & \omega^1_{2m} \\ \vdots & & \ddots & \vdots \\ \omega^1_{n1}, & \omega^1_{n2}, & \cdots, & \omega^1_{nm} \end{vmatrix} \tag{2.5.2}$$

The matrix multiplication of the input matrix and the weight matrix gives us a matrix of dimensions $1 \times m$. This correlates to the amount of nodes in the first hidden layer, a sum of weights multiplied by their corresponding input data for each hidden node.

$$\text{Matrix } \mathbf{I} \cdot \omega^1 \rightarrow [1 \times n] \cdot [n \times m] \rightarrow [1 \times m] : \mathbf{I} \cdot \omega = \begin{vmatrix} \mathbf{I}_1 \cdot \omega^1_{11} + \cdots + \mathbf{I}_n \cdot \omega^1_{n1}, & \cdots, \mathbf{I}_m \cdot \omega^1_{1m} + \cdots + \mathbf{I}_n \cdot \omega^1_{nm} \end{vmatrix} \tag{2.5.3}$$

Each node in the hidden layer will have a bias term β. Formatting this as a matrix, we get a $1 \times m$ matrix, which aligns with the dimensions matrix $\mathbf{I} \cdot \omega$ (2.5.3).

$$\text{Matrix } \beta^1 \rightarrow [1 \times m] : \beta^1 = \begin{vmatrix} \beta^1_1, & \beta^1_2, & \cdots, & \beta^1_m \end{vmatrix} \tag{2.5.4}$$

We apply an element-wise summation of the matrix $\mathbf{I} \cdot \omega$ and the bias matrix β, to find the matrix $\mathcal{S}^1 \rightarrow [1 \times m]$: the summation of each node in layer 1. Applying the activation function $\mathcal{F}_1$ to the summation matrix $\mathcal{S}$ gives us the output α of the hidden layer.

Note that the activation function is labeled $\mathcal{F}_1$ as it is the first activation function in the network, and can differ from other activation functions in the network.

$$\text{Matrix } \alpha \rightarrow [1 \times m] : \alpha = \mathcal{F}_1(\mathcal{S}^1) = \mathcal{F}_1(\mathbf{I} \cdot \omega^1 + \beta^1) \tag{2.5.5}$$

$$\text{where } \mathcal{S}^1_m = \sum_{k=1}^{n} \left(\mathbf{I}_k \cdot \omega^1_{km} \right) + \beta^1_m \tag{2.5.6}$$

This activation function is applied to every element in the matrix. This matrix α is then used as the input for the next hidden layer of the network, repeating the same process as before all the way to the output layer:

The weight matrix $\omega^{\mathcal{L}}$ is a $n \times m$ matrix, with each row representing the weights of a hidden node (n) in layer $\mathcal{L}-1$ to each connected node(m) in layer $\mathcal{L}$.

$$\text{Matrix } \omega^{\mathcal{L}} \rightarrow [n \times m] : \omega^{\mathcal{L}} = \begin{vmatrix} \omega_{11}^{\mathcal{L}}, & \omega_{12}^{\mathcal{L}}, & \cdots, & \omega_{1m}^{\mathcal{L}} \\ \omega_{21}^{\mathcal{L}}, & \ddots & & \omega_{2m}^{\mathcal{L}} \\ \vdots & & \ddots & \vdots \\ \omega_{n1}^{\mathcal{L}}, & \omega_{n2}^{\mathcal{L}}, & \cdots, & \omega_{nm}^{\mathcal{L}} \end{vmatrix} \tag{2.5.7}$$

(2.5.8)

$$\text{Matrix } \beta^{\mathcal{L}} \rightarrow [1 \times m] : \beta^{\mathcal{L}} = \begin{vmatrix} \beta_1^{\mathcal{L}}, & \beta_2^{\mathcal{L}}, & \cdots, & \beta_m^{\mathcal{L}} \end{vmatrix} \tag{2.5.9}$$

for l hidden layers.

The same summation applies to the hidden layer output α and the weight matrix ω for each hidden layer, all the way to the output layer. We travel layer by layer to reach our predicted output, O_q.

$$\text{Matrix } \alpha^{\mathcal{L}} \rightarrow [1 \times m] : \alpha^{\mathcal{L}} = \mathcal{F}_{\mathcal{L}}(\mathcal{S}^{\mathcal{L}}) = \mathcal{F}_{\mathcal{L}}\left(\alpha^{\mathcal{L}-1} \cdot \omega^{\mathcal{L}} + \beta^{\mathcal{L}}\right) \tag{2.5.10}$$

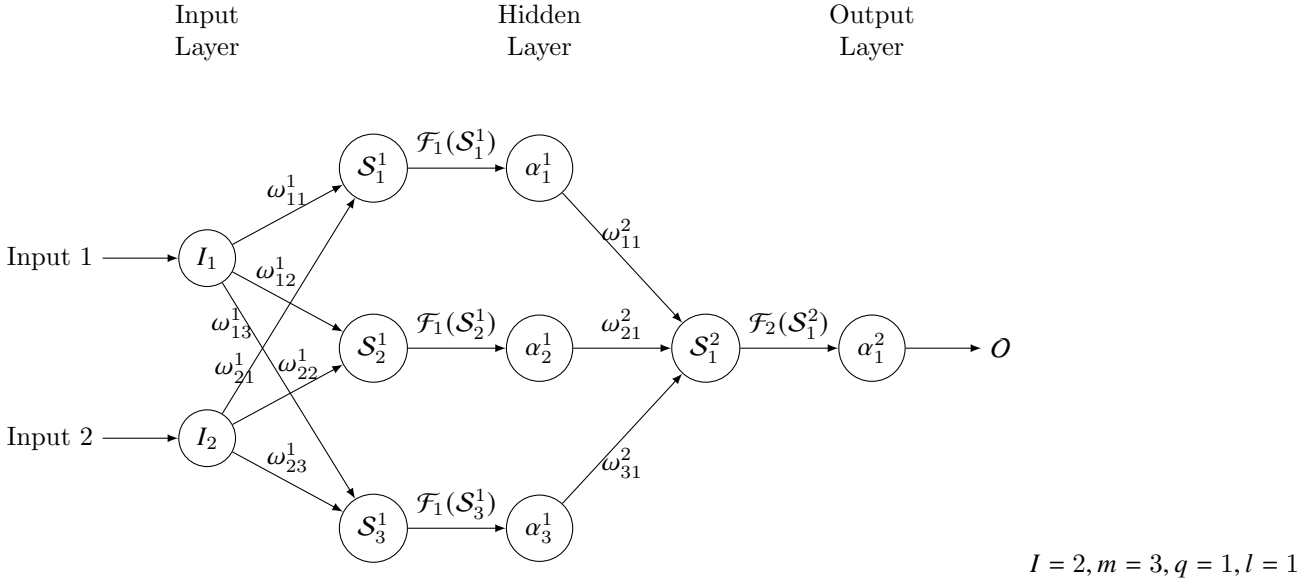
$$\text{where } \mathcal{S}_m^{\mathcal{L}} = \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}}\right) + \beta_m^{\mathcal{L}} \tag{2.5.11}$$

Note that $\mathcal{F}_{\mathcal{L}}$ is the activation function at layer $\mathcal{L}$.
Finally, we calculate the output of the network, O_q.

$$\text{Output } \rightarrow [1 \times q] : O_q = \mathcal{F}_{\mathcal{L}}(\mathcal{S}^{\mathcal{L}}) = \mathcal{F}_{\mathcal{L}}(\alpha^{\mathcal{L}-1} \cdot \omega^{\mathcal{L}} + \beta^{\mathcal{L}}) \tag{2.5.12}$$

This gives us our prediction from the forward pass through the network.

An example of a network with 1 hidden node is given below:



$I = 2, m = 3, q = 1, l = 1$

2.6 Backward Pass

The backward pass is used to update the weights and biases of the network, to reduce the error of the network and improve the accuracy of the model. Back propagation is about finding how much of the error is due to each weight in the network, and then updating the weights to reduce the error.

Leibniz's notation of the chain rule is used to find the partial derivative of the error with respect to the weights and biases in the network. Leibniz's notation states that *If a variable z depends on the variable y, which itself depends on the variable x (that is, y and z are dependent variables), then z depends on x as well, via the intermediate variable y.*

$$\frac{\delta z}{\delta x} = \frac{\delta z}{\delta y} \cdot \frac{\delta y}{\delta x} \tag{2.6.1}$$

To back propagate through the network, we need to find how much of the error (cost C) is due to each node in the network so that we can update the weights and biases to reduce the cost for future predictions. Using the chain rule, we can find the rate of change of the error with respect to the weights and biases in the network, working backwards from the output to each node. We can use partial derivatives to find the rate of change of the error with respect to each weight and bias in the network, by differentiating the weights and biases dependent variables.

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \cdot \frac{\delta \mathcal{S}_m^{\mathcal{L}}}{\delta \omega_{nm}^{\mathcal{L}}} \qquad \frac{\delta C}{\delta \beta_{nm}^{\mathcal{L}}} = \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \cdot \frac{\delta \mathcal{S}_m^{\mathcal{L}}}{\delta \beta_{nm}^{\mathcal{L}}} \tag{2.6.2}$$

Using the equation for $\mathcal{S}$ that we found in (2.5.11), we can differentiate the summation with respect to the weights and biases in the network:

$$\mathcal{S}_m^{\mathcal{L}} = \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad \rightarrow \frac{\delta}{\delta \omega_{nm}^{\mathcal{L}}} \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad = \alpha_n^{\mathcal{L}-1} \tag{2.6.3}$$

$$\mathcal{S}_m^{\mathcal{L}} = \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad \rightarrow \frac{\delta}{\delta \beta_m^{\mathcal{L}}} \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad = 1 \tag{2.6.4}$$

Substituting into (2.6.2):

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \qquad \rightarrow \quad \alpha_n^{\mathcal{L}-1} \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \cdot \frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} \tag{2.6.5}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = 1 \cdot \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \qquad \rightarrow \quad 1 \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \cdot \frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} \tag{2.6.6}$$

Then it follows with equation (2.5.10):

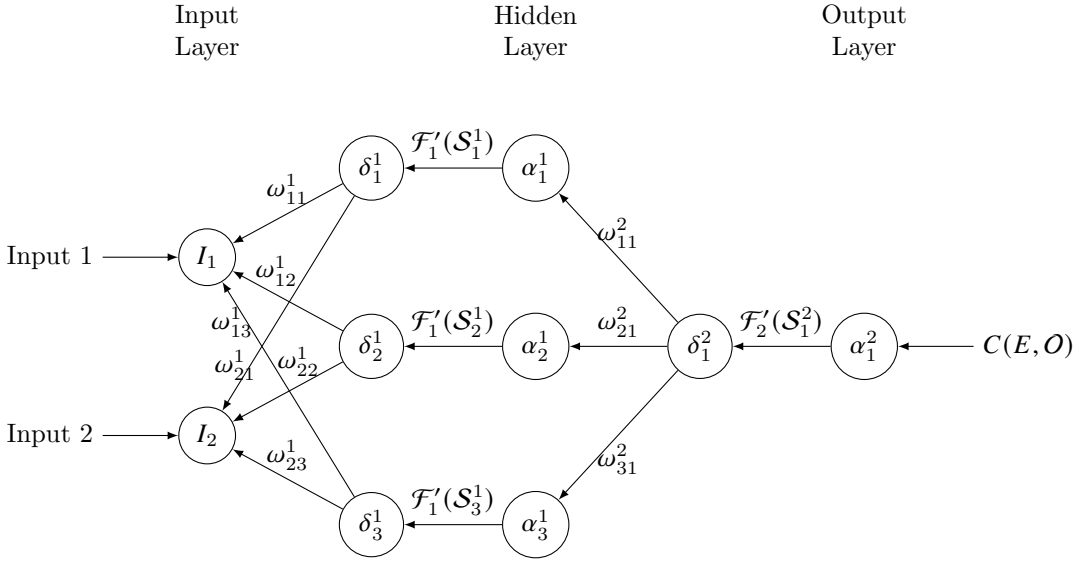
$$\frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} = \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \qquad \rightarrow \frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \tag{2.6.7}$$

$$\rightarrow \frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \tag{2.6.8}$$

As we are moving backwards through the network, we use the same theory that the gradient at the node is the sum of the gradients of the weights it is connected to, Where are influenced by the gradient at the node it is connected to. We call this value δ.

$$\frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} = \sum_{p=1}^{q} \left(\frac{\delta C}{\delta \omega_{mp}^{\mathcal{L}+1}} \right) = \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.9}$$

Referencing to our example network:



$I = 2, m = 3, q = 1, l = 1$

This sum is the result of propagating error back through the network and is the backbone of the back propagation

algorithm. Finally substituting this back into (2.6.7) and (2.6.8) we get the following:

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.10}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.11}$$

Using what we derived, we can find a formula for the delta values of the nodes in the network, collecting like terms from the equations above(2.6.10), (2.6.11).

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \delta_m^{\mathcal{L}} \tag{2.6.12}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \delta_m^{\mathcal{L}} \tag{2.6.13}$$

These delta values can be used in the form of matrices in our network to update the weights and biases. For the output layer, the delta value is the output of the cost function derivative, as 100% of the error comes through the output layer, and then we apply the derivative of the activation function.

$$\begin{gathered} \delta_m^{\mathcal{L}} = C(E_m, O_m) \cdot \mathcal{F}'_m(\mathcal{S}_m^{\mathcal{L}}) \\ \text{Matrix } \delta_m^{\mathcal{L}} \to [1 \times m] : \delta_m^{\mathcal{L}} = \left| C(E_1, O_1) \cdot \mathcal{F}'_1(\mathcal{S}_1^{\mathcal{L}}), \quad \cdots, \quad C(E_m, O_m) \cdot \mathcal{F}'_m(\mathcal{S}_m^{\mathcal{L}}) \right| \end{gathered} \tag{2.6.14}$$

Where $C(C_m, O_m)$ is the cost function derivative, E_m is the expected output and O_m is the predicted output of the node.

We can then construct an equation for the delta values of the hidden nodes in the network.

For the hidden nodes in layers $1 \leq k < \mathcal{L}$*:*

$$\begin{gathered} \delta_n^k = \mathcal{F}'_k(\mathcal{S}_n^k) \cdot \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \\ \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \to \text{ Matrix } [1 \times p] \cdot [n \times p]^T \to [1 \times n] \\ \text{Matrix } \delta_n^k \to [1 \times n] \circ [1 \times n] \to [1 \times n] : \delta_n^k = \left| \mathcal{F}'_k(\mathcal{S}_1^k) \cdot \textstyle\sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{1p}^{k+1} \right), \cdots, \mathcal{F}'_k(\mathcal{S}_n^k) \cdot \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \right| \end{gathered} \tag{2.6.15}$$

With n nodes in layer k, p nodes in layer $k+1$ and ω_{nq}^{k+1} being the weight between node n in layer k and node q in layer $k+1$. With $\circ$ being the element-wise multiplication of the two matrices.

Using our findings from equation (2.6.10) and (2.6.11), we can update the weights and biases in the network. We can update the weights and biases in the network by adding the gradient, multiplied by the learning rate η. This decides how much of the gradient we want to apply to the weights and biases for our next forward pass.

$$\text{Matrix } \omega_{nq}^k \to [n \times q] + [1 \times n]^T \cdot [1 \times q] \to [n \times q] : \omega'^k_{nq} = \omega_{nq}^k + \eta \cdot \alpha_n^{k-1} \cdot \delta_q^k \tag{2.6.16}$$

$$\text{Matrix } \beta'^k_{nq} \to [n \times q] + [1 \times q] \to [n \times 1] : \beta'^k_{nq} = \beta_{nq}^k + \eta \cdot \delta_q^k \tag{2.6.17}$$

It is worth remembering that we only update the weights and biases after we have calculated all the delta values for the network, as the error of the network is dependent on what these values were when the forward pass was made. Once we calculate the delta values for the network, we can then update the weights and biases.

2.7 Implementation of a single hidden layer network

A single hidden layer can be implemented with a defined structure. As we know how many nodes and layers there are, we can hard code the dimensions. We can initialise the weights and biases randomly, with the structure that we define.

back_prop_single.py

```
import numpy as np

# Define the structure of the network
perceptronStructure = (7, 16, 1)

```

```
    def init_structure(perceptronStructure):
        '''Intialises the perceptron structure with random weights'''

        # Weights and biases for input layer to the hidden layer
        hiddenWeightMatrix = np.random.uniform(-1, 1, (perceptronStructure[0], perceptronStructure[1]))
        hiddenBiasMatrix = np.random.uniform(-1, 1, (1, perceptronStructure[1]))

        # Weights and biases for hidden layer to the output layer
        outputWeightMatrix = np.random.uniform(-1, 1, (perceptronStructure[1], perceptronStructure[2]))
        outputBiasMatrix = np.random.uniform(-1, 1, (1, perceptronStructure[2]))

        # Create lists for input and output
        hiddenList = [hiddenWeightMatrix, hiddenBiasMatrix]
        outputList = [outputWeightMatrix, outputBiasMatrix]

        return hiddenList, outputList


    hiddenList, outputList = init_structure(perceptronStructure)
```

In this example, we have a network with 7 input nodes, 16 hidden nodes and 1 output node.

We iterate through each of the data points and pass them through the forward pass of the network. We pass them through a scaling function to normalise the data, then pass them through the network. By doing this, the network can appropriately learn to predict the output as all datapoints are are passed through an activation function that outputs between 0 and 1.

back_prop_single.py

```
    # Normalization function
    def min_max_scaler(data):
        '''Function to normalize the data'''
        return (data - data.min()) / (data.max() - data.min())

    def min_max_reverser(data, dataMax, dataMin):
        '''Function to normalize the data'''
        return data * (dataMax - dataMin) + dataMin


    def forward_pass(data, hiddenList, outputList):
        '''Forward pass of the neural network'''

        # Calculate for the hidden layer
        hiddenSum = (inputData @ hiddenList[0]) + hiddenList[1]
        hiddenActivated = sigmoid(hiddenSum)

        # Calculate for the output layer
        outputSum = (hiddenActivated @ outputList[0]) + outputList[1]
        outputActivated = sigmoid(outputSum)

        return hiddenSum, hiddenActivated, outputSum, outputActivated


    while count < epochs:

        # Iterate through the training data
        for i, day in enumerate(trainingData.itertuples(index=False), start=0):

            inputData = np.array([float(day[2]), float(day[3]), float(day[4]), float(day[5]), float(day[6]), float(day[7]), float(day[8])])
            expectedOutput = np.array([float(day[1])])

            # Forward pass
            hiddenSum, hiddenActivated, outputSum, outputActivated = forward_pass(inputData, hiddenList, outputList)
```

We can then expand by introducing the backward pass to update the weights and biases of the network.

back_prop_single.py

```
    def backward_pass(expectedOutput, hiddenSum, hiddenActivated, outputSum, outputActivated, hiddenList, outputList, inputData, dfLength):
        '''Backward pass of the neural network'''

        # Calculate the cost of the perceptron structure
        structureCost = cost_function_derivative(expectedOutput, outputActivated, dfLength)

        # Calculate the delta values for the output layer
        outputDelta = structureCost * sigmoid_derivative(outputSum)

```

```
        # Calculate the delta values for the hidden layer
        hiddenDelta = (outputDelta @ outputList[0].T) * sigmoid_derivative(hiddenSum)


        # Update the weights and biases for the output layer
        outputList[0] += (hiddenActivated.T @ outputDelta) * learningRate
        outputList[1] += outputDelta * learningRate

        # Update the weights and biases for the hidden layer
        hiddenList[0] += (inputData.T @ hiddenDelta) * learningRate
        hiddenList[1] += hiddenDelta * learningRate

        return (hiddenList, outputList)


    while count < epochs:

        # Iterate through the training data
        for i, day in enumerate(trainingData.itertuples(index=False), start=0):

            inputData = np.array([float(day[2]), float(day[3]), float(day[4]), float(day[5]), float(day[6]), float(day[7]), float(day[8])])
            expectedOutput = np.array([float(day[1])])

            # Forward pass
            hiddenSum, hiddenActivated, outputSum, outputActivated = forward_pass(inputData, hiddenList, outputList)
            hiddenList, outputList = backward_pass(expectedOutput, hiddenSum, hiddenActivated, outputSum, outputActivated, hiddenList, outputList, inputData.reshape(1, -1), dfLength)

        count += 1
```

We can then create predictions after a given amount of epochs, and calculate the error of the network. We can then plot these predictions against the expected output to see how well the network is performing.

A single layer network does not allow for complex patterns to be learned, so to improve the network, we can add more hidden layers to the network. As stated previously, repeatedly using the sigmoid activation over multiple layers can cause the gradient to vanish, so we can allow the activation to be defined for each indeviual layer.

2.8 Multiple Hidden Layers

To implement multiple hidden layers, we can define the structure of the network as a tuple. The first element of the tuple is the number of input nodes, and the second element is a list of tuples, each with the amount of nodes and their activation function. We can iteratively generate hidden layers and then use the same process to calculate changes and weight updates.

back_prop_multi.py

```
    import numpy as np

    # Define the structure of the network
    perceptronStructure = (7, [(16, 'sigmoid'),(16, 'sigmoid'), (1, 'sigmoid')])

    # Initialise Perceptron Structure
    def init_structure(perceptronStructure):
        '''Initialises the perceptron structure'''

        # Split structure
        inputDimensions = perceptronStructure[0]
        nodeDimensions = perceptronStructure[1]

        # Create lists for weights and biases
        weightList = []
        biasList = []

        # Initialise the first layer of the perceptron
        weightMatrix = np.random.uniform(-1, 1, (inputDimensions, nodeDimensions[0][0]))
        biasMatrix = np.random.uniform(-1, 1, (1, nodeDimensions[0][0]))

        # Append to the lists
        weightList.append(weightMatrix)
        biasList.append(biasMatrix)

        # Initialise the node layers of the perceptron
        for i in range(1, len(nodeDimensions)):
            weightMatrix = np.random.uniform(-1, 1, (nodeDimensions[i - 1][0], nodeDimensions[i][0]))
            biasMatrix = np.random.uniform(-1, 1, (1, nodeDimensions[i][0]))

```

```
            # Append to the lists
            weightList.append(weightMatrix)
            biasList.append(biasMatrix)

        return weightList, biasList
```

As each layer can have its own activation function, we can define the activation function for each layer. To do this, each layer has a tuple with the number of nodes and the activation function. These are then stored as a list and joined with the input dimensions to create the structure of the network.

The structure we use to iterate through the layers is the same as our backwards pass. We calculate our first item, and then iterate through the rest of the layers until we reach the end.

However, for the forward pass, we just go through the whole list and calculate the forward pass for each layer iteratively. This is because it is simpler and each step is the same computation

back_prop_multi.py

```
    import numpy as np

    layerStructure = perceptronStructure[1]

    def forward_pass(inputData, weightList, biasList, layerStructure):
        '''Forward pass of the neural network'''

        # Create a list for the activated nodes and summed nodes
        summedList = []
        activatedList = []


        # Calculate for the hidden layers
        for layer, structure in enumerate(layerStructure):
            # Get activation function
            activationFunction = get_activation(structure[1])

            # Calculate the sum of the matrix
            sumMatrix = (inputData @ weightList[layer]) + biasList[layer]

            activatedMatrix = activationFunction(sumMatrix)

            # Append to the list
            summedList.append(sumMatrix)
            activatedList.append(activatedMatrix)

            # Update the input data
            inputData = activatedMatrix

        return activatedList, summedList
```

The backward pass is the same as the single layer network, but we iterate through the layers and calculate the delta values for each layer. We calculate the output layer first and then iterate through the hidden layers to calculate the delta values for each layer. Then, we do the same process for updating the weights and biases of the network. We do 1 layer separately as the output layer is different to the hidden layers, and the first hidden layer is different to the rest of the hidden layers in terms of calcualtions.

back_prop_multi.py

```
    def backward_pass(expectedOutput, weightList, biasList, summedList, activatedList, inputData,
    layerStructure):
        '''Backward pass of the neural network'''

        # Cost of the network for that pass
        structureCost = cost_function_derivative(expectedOutput, activatedList[-1])

        # Calculate the delta values for the output layer
        activation_derivative = get_activation_derivative(layerStructure[-1][1])
        outputDelta = structureCost * activation_derivative(summedList[-1])

        # Calculate the delta values for the hidden layers
        deltaList = []
        deltaList.append(outputDelta)

        # Calculate the delta values for the hidden layers
        for layer in range(2, len(weightList)+1, 1):
            activation_derivative = get_activation_derivative(layerStructure[-layer][1])
            delta = (deltaList[-1] @ weightList[-(layer)+1].T) * activation_derivative(summedList[-layer])
            deltaList.append(delta)

        # Reverse the delta list
        deltaList.reverse()
```

```
25
26          # Calculate the weights and biases with input layer as previous layer
27          weightList[0] += (inputData.T @ deltaList[0]) * learningRate
28          biasList[0] += deltaList[0] * learningRate
29
30
31          # Update the weights and biases for the output layer
32          for layer in range(1, len(weightList), 1):
33              weightList[layer] += (activatedList[layer-1].T @ deltaList[layer]) * learningRate
34              biasList[layer] += deltaList[layer] * learningRate
35
36          return (weightList, biasList)
```

The format of data remains the same and we can train the network in the same way as the single layer network. We can then plot the results and see how well the network is performing.

The main downfall of this implementation is the speed of executing the algorithm. The algorithm is not optimised for speed, and the calculations are done in a way that is not efficient. This can be helped in a later version of the code.

We can plot some of the results of the network to see how well it is performing.

back_prop_multi.py

```
2
3       perceptronStructure = (7, [(16, 'sigmoid'),(16, 'sigmoid'), (1, 'sigmoid')])
4       learningRate = 0.1
5       epochs = 1000
```

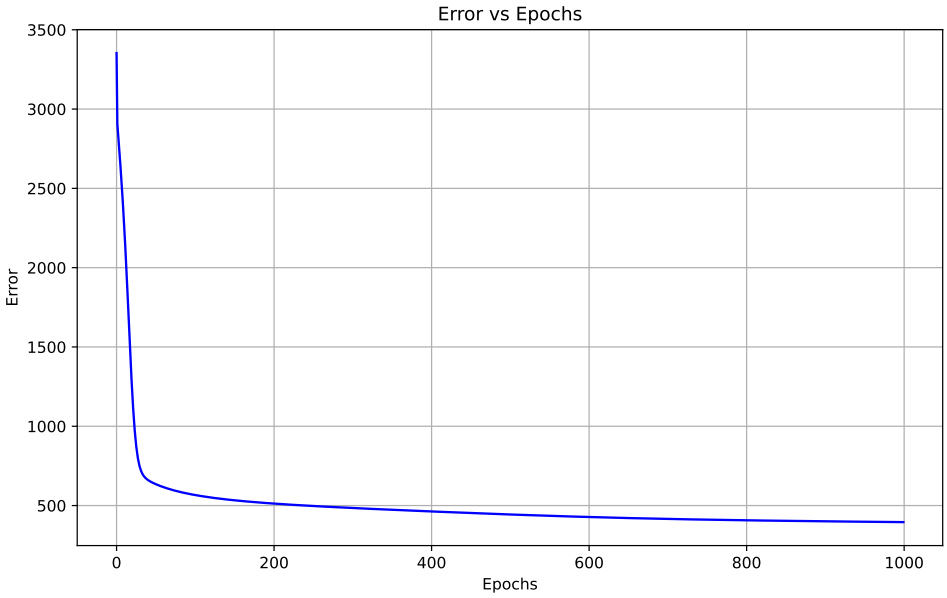


Figure 2.8.1: Error vs. Epochs

This graph shows the average error of the network over the number of epochs. The error was calculated with the mean squared error function. The error decreases as the number of epochs increases, showing that the network is learning the patterns in the data. The final MSE at epoch 1000 is about 410. This means the RMSE is about 20.25. This interpretation is that the network is off by about 20.25 meters per second cubed on average off its true value.

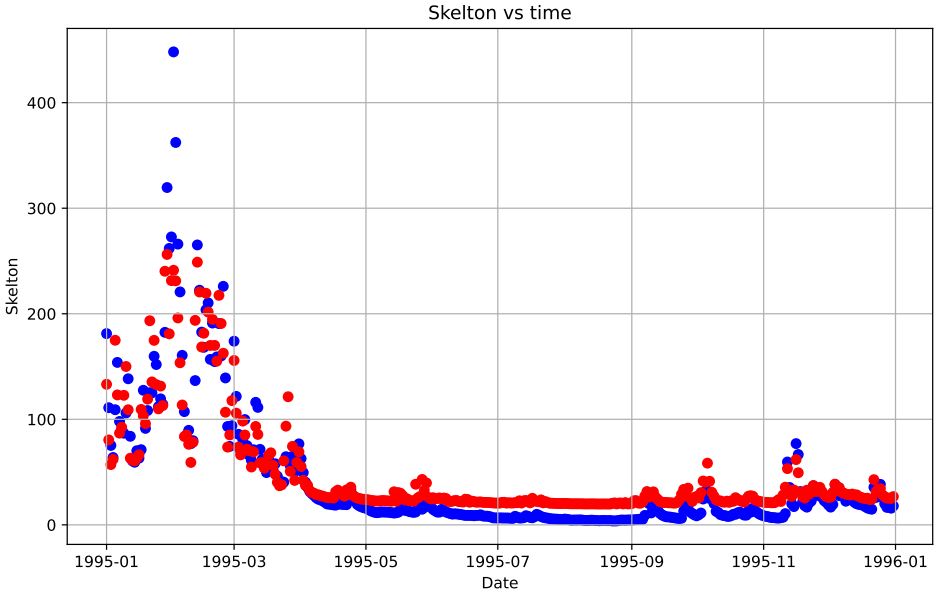


Figure 2.8.2: Predicted vs. Expected Output

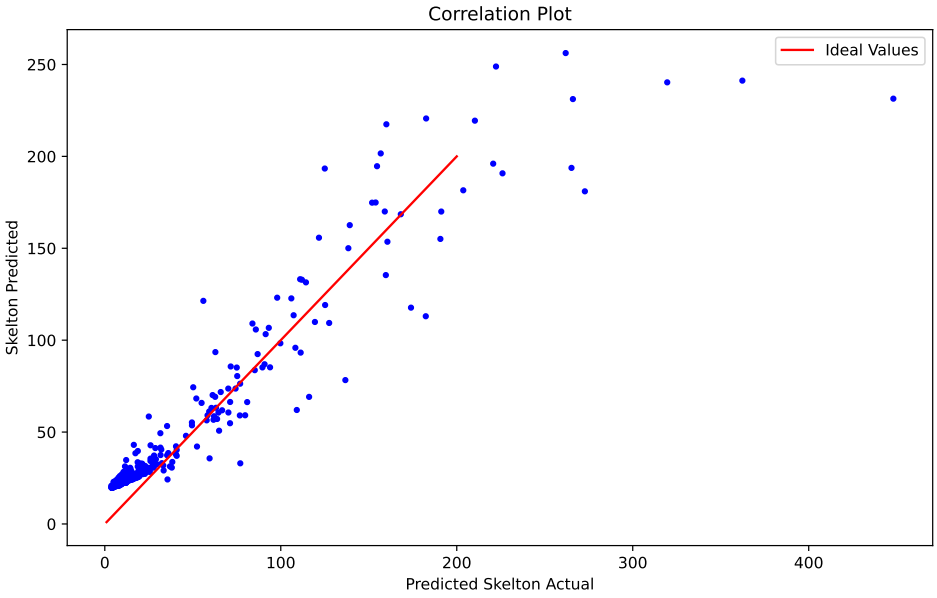


Figure 2.8.3: Correlation of Predicted vs. Expected Output

back_prop_multi.py

```
    perceptronStructure = (7, [(16, 'sigmoid'),(16, 'sigmoid'), (1, 'sigmoid')])
    learningRate = 0.1
    epochs = 10000
```

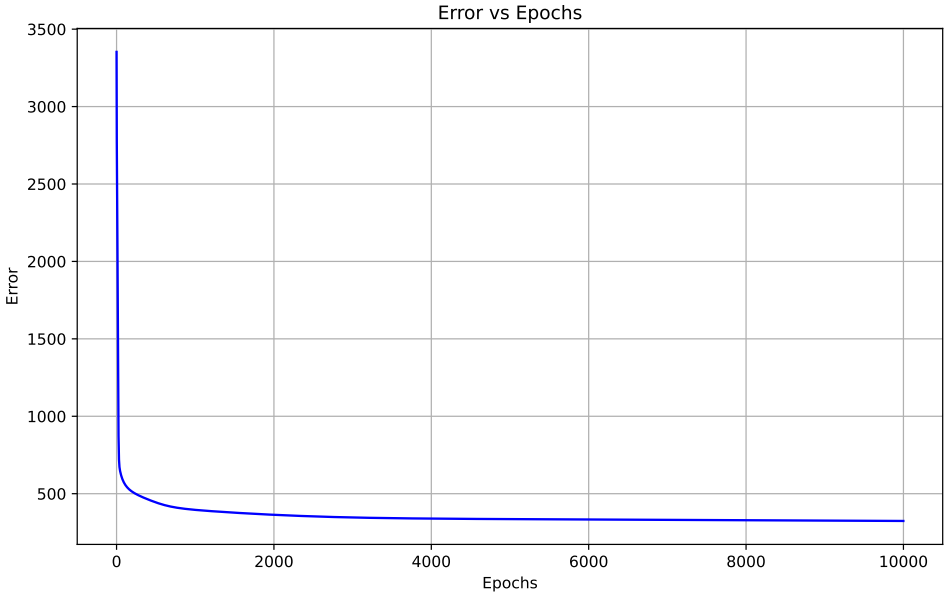


Figure 2.8.4: Error vs. Epochs

The MSE in this graph is around 250, which means the RMSE is about 15.8. This provides a much better result than the previous network, showing that the network is learning the patterns in the data. However, we can see that the network stopped learning after about 5000 epochs, as the error does not decrease after this point. We can alter the structure of the network and improve it to reduce the errors

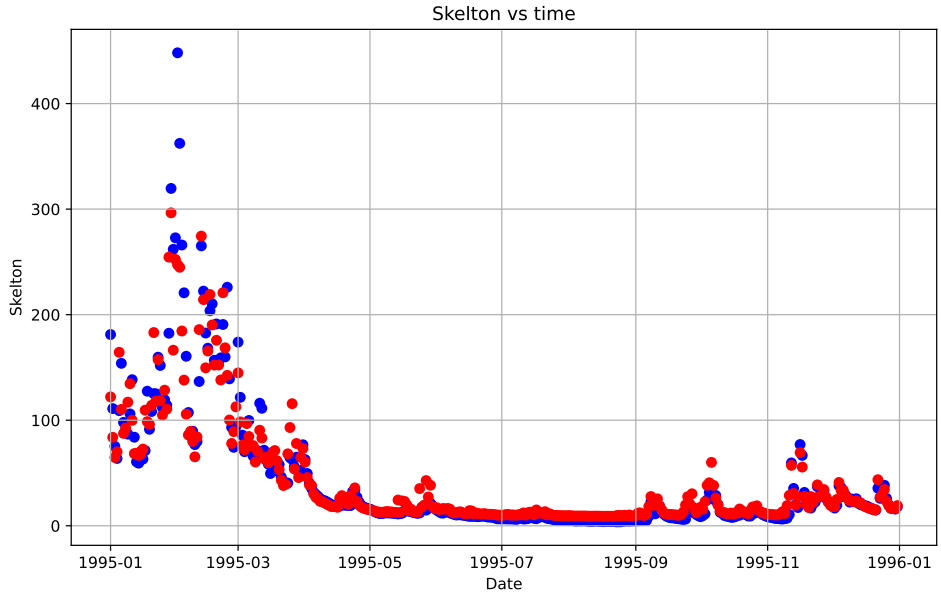


Figure 2.8.5: Predicted vs. Expected Output

Figure 2.6.5 clearly shows a better and more accurate prediction of the network compared to figure 2.6.2.

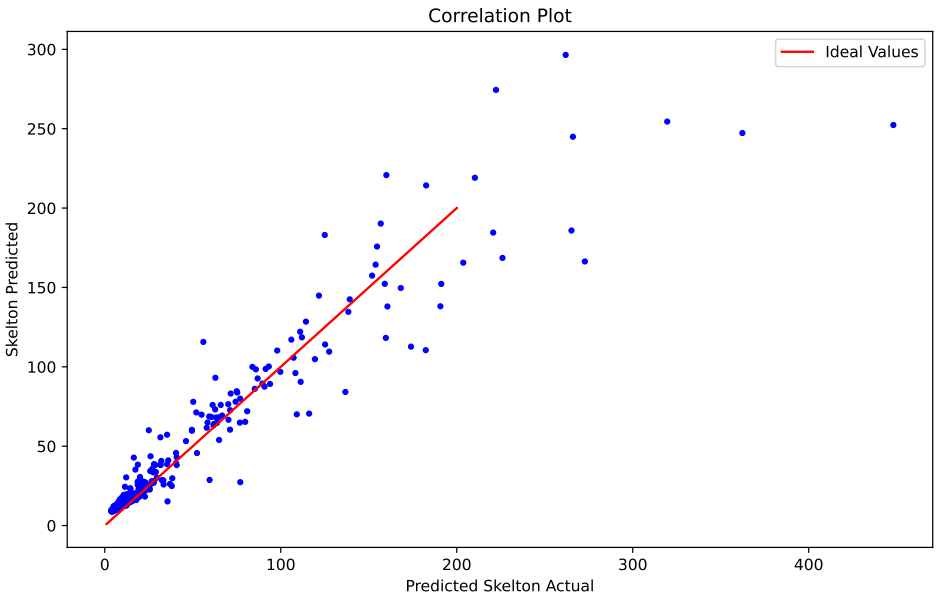


Figure 2.8.6: Correlation of Predicted vs. Expected Output

The correlation of the network in figure 1.6.6 is much better than the correlation in figure 1.6.3. The data is less skewed and the network is predicting the data more accurately.

We can see that the network is learning the patterns in the data, but we can improve the network by changing the structure of the network. To improve the data, we can code improvements to the network to make it more efficient and faster. However, the code is slow to execute and the speed of the implementation needs to improved to make it more efficient. There are also datapoints that are clearly outliers too, and these can be removed to improve the network. Once these points are removed we may see a higher accuracy in the network.

2.9 Object oriented implementation

As previously mentioned, the data has many outliers that are affecting the network, and these need to be removed. The previous parameters were:

```
data_cleaning.py

    iqrWeight = 1.5
    sdWeight = 2
    windowSize = 10
    replacementRange = 6
```

And these will be changed to:

```
data_cleaning.py

    iqrWeight = 1.4
    sdWeight = 1.6
    windowSize = 20
    replacementRange = 8
```

This removes 1559 data points from the dataset, which is 150% more than the previous parameters.

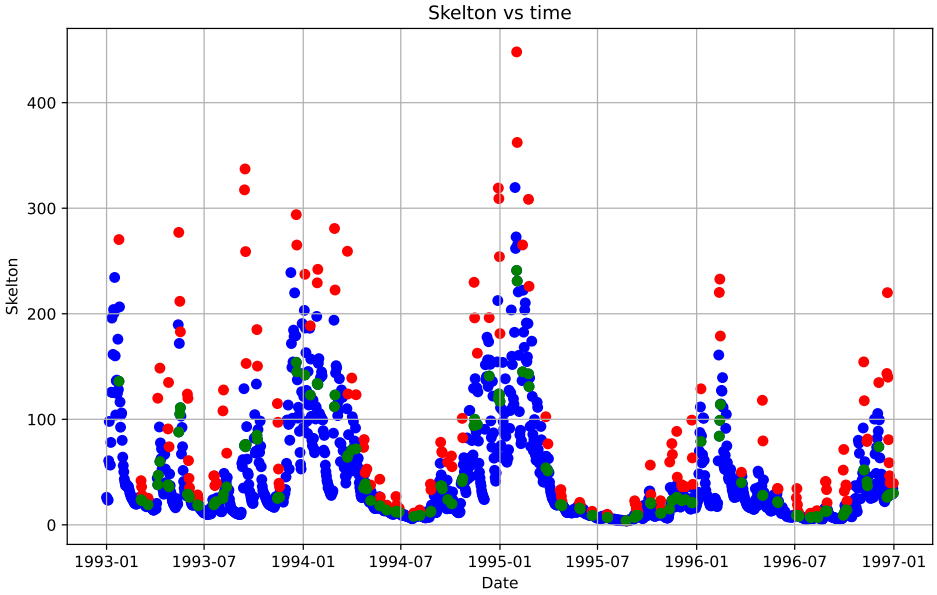


Figure 2.9.1: Cleaned Skelton Data

We can then implement the network in an object-oriented way to make it more efficient and easier to use.

back_prop_OO.py

```
import numpy as np
import pandas as pd

class MLP:

    def __init__(self, dataSet:pd.DataFrame, nodeStructure:list[tuple], learningRate:float, trainingEpochs:int, PredictedColumn:str):
        '''Initialises the perceptron structure'''

        # Determine the number of input dimensions
        self.inputDimensions = (dataSet.shape[1]-2)

        # Normalise the training data
        self.dataFrameMin = dataSet[PredictedColumn].min()
        self.dataFrameMax = dataSet[PredictedColumn].max()
        self.dataSet = dataSet.copy()
        self.dataSet.iloc[:, 1:] = self.min_max_scaler(self.dataSet.iloc[:, 1:])

        # Split the data into training and testing
        self.trainingData = self.dataSet[dataSet['DATE'].dt.year.isin([1993, 1994])]
        self.testingData = self.dataSet[dataSet['DATE'].dt.year == 1995]


        # Store the structure of the network
        self.nodeStructure = nodeStructure
        self.learningRate = learningRate
        self.trainingEpochs = trainingEpochs

        # Create lists for weights and biases
        self.weightList = []
        self.biasList = []

        # Populate the structure lists
        previousLayer = self.inputDimensions

        for layer in self.nodeStructure:
            # Create matrices for the weights and biases
            weightMatrix = np.random.uniform(-1, 1, (previousLayer, layer[0]))
            biasMatrix = np.random.uniform(-1, 1, (1, layer[0]))

            # Append to the lists
            self.weightList.append(weightMatrix)
            self.biasList.append(biasMatrix)

            # Update the previous layer
            previousLayer = layer[0]

```

```
            # Create dataframe for storing the error per epoch
            self.epochErrorDF = pd.DataFrame(columns=['Epoch', 'Error'])

            # Create dataframe for storing the predictions
            self.predictionDF = pd.DataFrame(columns=['DATE', 'Skelton'])


        def train_network(self) -> pd.DataFrame:
        '''Function to train the neural network'''

        # Iterate through the training epochs
        for epoch in range(self.trainingEpochs):

            # Create a dataframe to store the mean error for the epoch
            meanErrorList = []

            # Convert data to numpy array
            inputDataArray = self.trainingData.iloc[:, 2:].values
            expectedOutputArray = self.trainingData.iloc[:, 1].values

            # Iterate though each day in the training data
            for i in range(len(inputDataArray)):

                # Split the data point into input and output
                inputData, expectedOutput = inputDataArray[i], expectedOutputArray[i]

                # Pass the data through the network
                activatedList, summedList = self.forward_pass(inputData)

                meanErrorList.append(self.error_storing(expectedOutput, activatedList[-1]))

                # Backward pass through the network
                self.backward_pass(expectedOutput, summedList, activatedList, inputData)

            # Convert errors into dataframe
            meanErrorDF = pd.DataFrame({'Error' : meanErrorList})

            # Calculate the mean error for the epoch
            meanError = meanErrorDF['Error'].mean()

            # Store the mean error for the epoch
            newError = pd.DataFrame({'Epoch': [epoch], 'Error': [meanError]})
            self.epochErrorDF = pd.concat([self.epochErrorDF, newError], ignore_index=True)

            # Print the error for the epoch
            if epoch % 100 == 0:
                print(f'Epoch: {epoch}, Error: {meanError}')
```

The object oriented approach should hopefully increase speeds. To be able to compare networks accurately, in terms of error and time taken, I will use the same machine and parameters to run the networks.

back_prop_OO.py

```
    import time

    # Define the structure of the network
    nodeStructure = [(16, 'sigmoid'),(16, 'sigmoid'), (1, 'sigmoid')]

    # Define the learning rate and epochs
    learningRate = 0.1
    trainingEpochs = 10000

    # Fix random seed for reproducibility
    np.random.seed(42)

    start_time = time.time()

    if __name__ == '__main__':
        main()

    end_time = time.time()
    print(f'Time taken: {end_time - start_time} seconds')
```

These same parameters will be used for the object-oriented implementation and the previous implementation.

Objected Orientated	Prodcedural
546	2154

Table 2.1: Time taken to run 10,000 epochs in seconds

The object-oriented implementation is much faster than the procedural implementation, with a speed increase of 75%.

This is a significant improvement and shows that the object-oriented implementation is much more efficient.

It is worth noting that the object-oriented implementation is not a perfect match in terms of each steps, as there were minor improvements made to the code to fix some issues. However, the main structure of the code is the same, and the improvements made to the code are minor, and both return the same results.

2.10 Batch Learning

Batch learning is an improvement meant to speed up the learning process of the network. Instead of updating the weights and biases after each data point, we can update the weights and biases after a batch of data points. This relys on more efficient matrix calculations, rather than iteration.

As well as offering an increase in speed computationaly, it can also offer an increase to the speed of learning (reduce the amount of epochs used). This is because by averaging the error over a batch of data points, we can reduce the noise in the data and get a more accurate representation of the error. This can help the network learn the patterns in the data more efficiently.

back_prop_batch_learning.py

```
    class MLP:

        def __init__(self, dataSet:pd.DataFrame, nodeStructure:list[tuple], learningRate:float, trainingEpochs:int, PredictedColumn:str):
            '''Initialises the perceptron structure'''

            # Determine the number of input dimensions
            self.inputDimensions = (dataSet.shape[1]-2)

            # Normalise the training data
            self.dataFrameMin = dataSet[PredictedColumn].min()
            self.dataFrameMax = dataSet[PredictedColumn].max()
            self.dataSet = dataSet.copy()
            self.dataSet.iloc[:, 1:] = self.min_max_scaler(self.dataSet.iloc[:, 1:])

            # Reshape into a 3D array
            self.dataSet = self.dataSet.to_numpy().reshape(dataSet.shape[0], 1, dataSet.shape[1])

            # Split the data into training and testing
            self.trainingData = self.dataSet[dataSet['DATE'].dt.year.isin([1993, 1994])]
            self.testingData = self.dataSet[dataSet['DATE'].dt.year == 1995]


            # Store the structure of the network
            self.nodeStructure = nodeStructure
            self.learningRate = learningRate
            self.trainingEpochs = trainingEpochs

            # Create lists for weights and biases
            self.weightList = []
            self.biasList = []

            # Populate the structure lists
            previousLayer = self.inputDimensions

            for layer in self.nodeStructure:
                # Create matrices for the weights and biases
                weightMatrix = np.random.uniform(-1, 1, (previousLayer, layer[0]))
                biasMatrix = np.random.uniform(-1, 1, (1, layer[0]))

                # Append to the lists
                self.weightList.append(weightMatrix)
                self.biasList.append(biasMatrix)

                # Update the previous layer
                previousLayer = layer[0]

            # Create dataframe for storing the error per epoch
            self.epochErrorDF = pd.DataFrame(columns=['Epoch', 'Error'])

            # Create dataframe for storing the predictions
            self.predictionDF = pd.DataFrame(columns=['DATE', 'Skelton'])

        def train_network(self) -> pd.DataFrame:
            '''Function to train the neural network'''

            # Iterate through the training epochs
            for epoch in range(self.trainingEpochs):
```

```
60
61                  # Split the data point into input and output, maintaining the 3D structure
62                  inputData, expectedOutput = self.trainingData[:, :, 2:], self.trainingData[:, :, 1:2]
63
64                  # Pass the data through the network
65                  activatedList, summedList = self.forward_pass(inputData)
66
67                  # Calculate the error of the network
68                  meanErrorList = self.error_storing(expectedOutput, activatedList[-1])
69
70                  # Reshape dimensions
71                  meanErrorList = meanErrorList.squeeze(axis=1)
72
73                  # Average the outputs
74                  meanErrorList = meanErrorList.mean(axis=1)
75
76                  # Backward pass through the network
77                  self.backward_pass(expectedOutput, summedList, activatedList, inputData)
78
79                  # Convert errors into dataframe
80                  meanErrorDF = pd.DataFrame({'Error' : meanErrorList})
81
82                  # Calculate the mean error for the epoch
83                  meanError = meanErrorDF['Error'].mean()
84
85                  # Store the mean error for the epoch
86                  newError = pd.DataFrame({'Epoch': [epoch], 'Error': [meanError]})
87                  self.epochErrorDF = pd.concat([self.epochErrorDF, newError], ignore_index=True)
88
89                  # Print the error for the epoch
90                  if epoch % 100 == 0:
91                      print(f'Epoch: {epoch}, Error: {meanError}')
```

I chose to implement this by using pythons solution to mismatched dimensions, broadcasting. This allows me to define all inputs and values at nodes (S, α) as 3D arrays. With broadcasting, I only need one weight matrix, saving space and computation time. This is because the weight matrix can be broadcasted to the correct dimensions for each layer.

When run with our testing structure:

Batch Learning	Objected Orientated
70	546

Table 2.2: Time taken to run 10,000 epochs in seconds

This time was acheived when passing the whole dataset as a batch. This is a significant improvement in speed, with a 87% increase in speed. This is a significant improvement and shows that the batch learning implementation is much more efficient. However, the batch learning with the whole batch has a much higher error than the previous implementations.

The batch learning implementation does not yield the same accuracy as prvious implementations due to the batch size. Large batch sizes remove patterns that can be learnt from, and the network can not evolve or learn from these patterns as they are smoothed out.

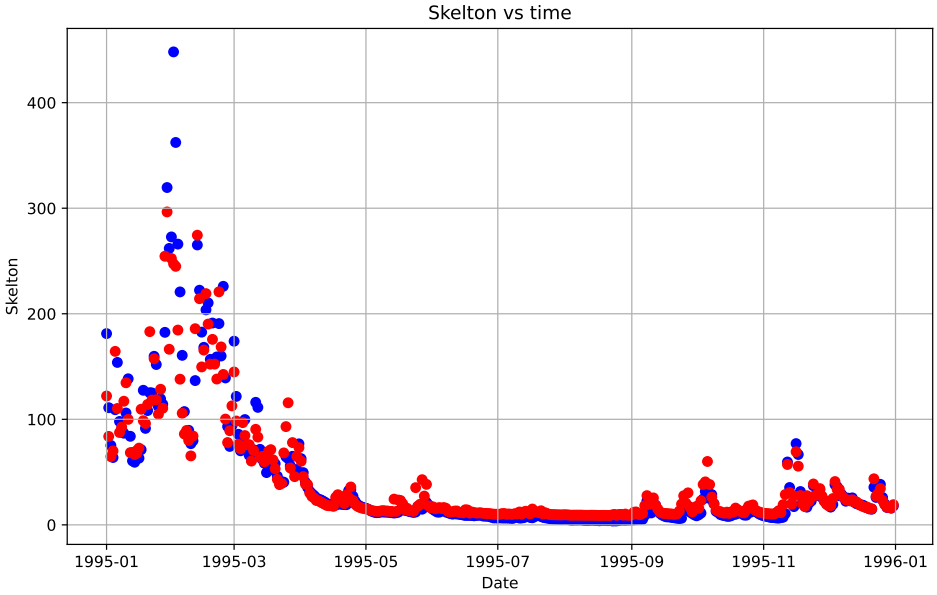


Figure 2.10.1: Predicted vs. Expected Output for non-Batrch Learning

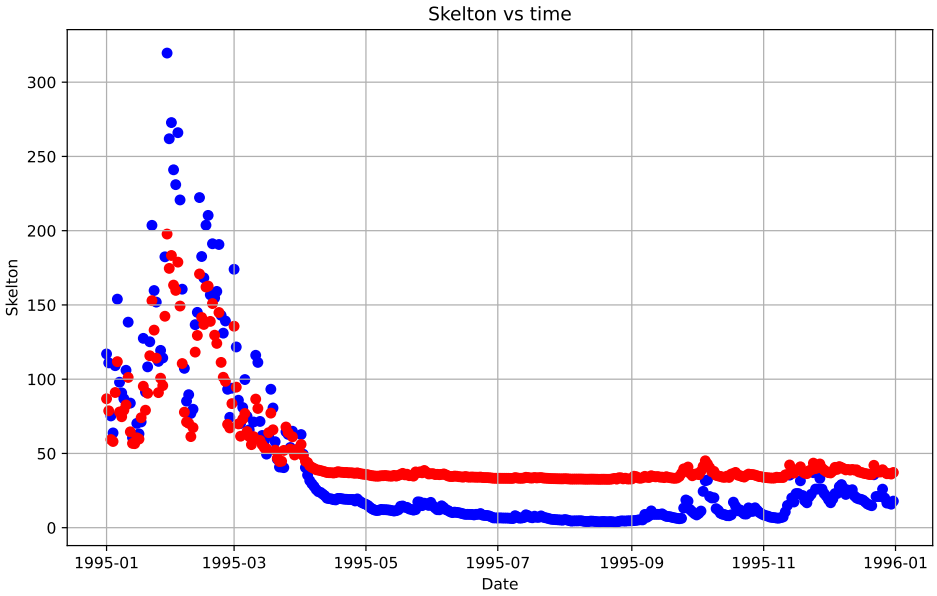


Figure 2.10.2: Predicted vs. Expected Output for Batrch Learning

As clearly visible in figures 2.6.8 and 2.6.9, the batch learning implementation is not as accurate as the non-batch learning implementation. We can however, alter the batch size, either statically or dynamically, therefor allowing for patters to be learnt from the data.

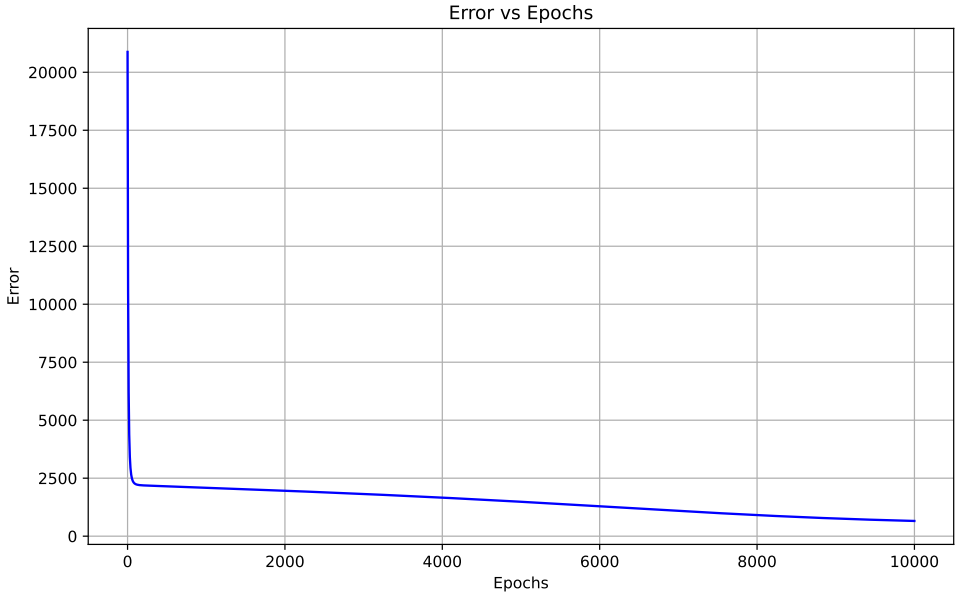


Figure 2.10.3: Error vs. Epochs for Batch Learning

Using data from this graph, we can try to calculate the error of the gradient, so that when it stagnates, we can reduce the batch size. This allows for fast learning before ending of accurate predictions.

back_prop_batch_learning.py

```
    class MLP:

    def train_network(self) -> pd.DataFrame:
        '''Function to train the neural network'''

        # Iterate through the training epochs
        for epoch in range(self.trainingEpochs):

            # Define the batch data
            batchData = np.array_split(self.trainingData, self.batchSize)

            # Define the mean error list
            meanErrorList = []

            # Iterate through the batches
            for batch in batchData:

                # Split the data point into input and output, maintaining the 3D structure
                inputData, expectedOutput = batch[:, :, 2:], batch[:, :, 1:2]

                # Pass the data through the network
                activatedList, summedList = self.forward_pass(inputData)

                # Calculate the error of the network
                meanErrorbatch = self.error_storing(expectedOutput, activatedList[-1])

                # Reshape dimensions
                meanErrorbatch = meanErrorbatch.squeeze(axis=1)

                # Average the outputs and store in the list
                meanErrorList.append(meanErrorbatch.mean())

                # Backward pass through the network
                self.backward_pass(expectedOutput, summedList, activatedList, inputData)


            # Convert errors into dataframe
            meanErrorDF = pd.DataFrame({'Error' : meanErrorList})

            # Calculate the mean error for the epoch
            meanError = meanErrorDF['Error'].mean()

            # Store the mean error for the epoch
            newError = pd.DataFrame({'Epoch': [epoch], 'Error': [meanError]})
            self.epochErrorDF = pd.concat([self.epochErrorDF, newError], ignore_index=True)

```

```
            # Only allow changes every 200 epochs to force the network to learn at certain points, and enforces at least 2 epochs
            if epoch % 200 == 0 and len(self.epochErrorDF) > 1:

                # Calculate the error gradient with respect to the batch size
                errorGradient = np.gradient(self.epochErrorDF['Error'])

                print(f'Error Gradient: {errorGradient[-1]}')

                # Doesn't allow batch size to be greater than the training data
                if self.batchSize == len(self.trainingData):
                    None

                # Increase the batch size
                elif abs(errorGradient[-1]) < 0.2:
                    self.batchSize = self.batchSize * 2

                    # Forces batch size to be the same as the training data
                    if self.batchSize > len(self.trainingData):
                        self.batchSize = len(self.trainingData)
```

This code allows for batch size to be dynamically decreased, so that when the error gradient stagnates, the batch size is reduced. This allows for the network to learn more efficiently and accurately.

Batch Learning	Batch Learning with Dynamic Batch Size	Objected Orientated
70	259	546

Table 2.3: Time taken to run 10,000 epochs in seconds

Batch Learning	Batch Learning with Dynamic Batch Size	Objected Orientated
665	295	250

Table 2.4: MSE at 10,000 epochs

At the 10,000 epoch mark, the dynamic batch size does not reach the same error as the standard object-oriented implementation. If trained for longer, it will reach the same point.

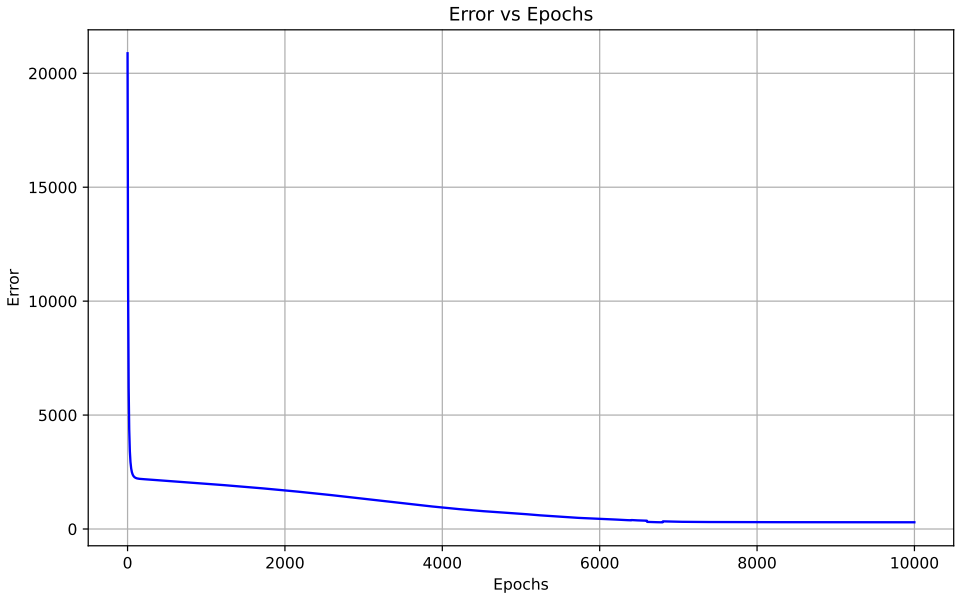


Figure 2.10.4: Error Gradient jumps in Dynamic Batch Size

However, the dynamic batch size implementation is much faster than the object-oriented implementation, with a 46% increase in speed. This is a significant improvement and shows that the dynamic batch size implementation is much more efficient. As the batch size approches the size of the training data, the error will decrease, and the network will learn slower. This means that for larger epochs, the speed increase is static, and not proportional to the amount of epochs. As epochs inceases, the difference in speed between the dynamic batch size and the object-oriented implementation will decrease, and so will the error. This implementation is best used for closing ground quickly and then fine-tuning the network with a smaller batch size.

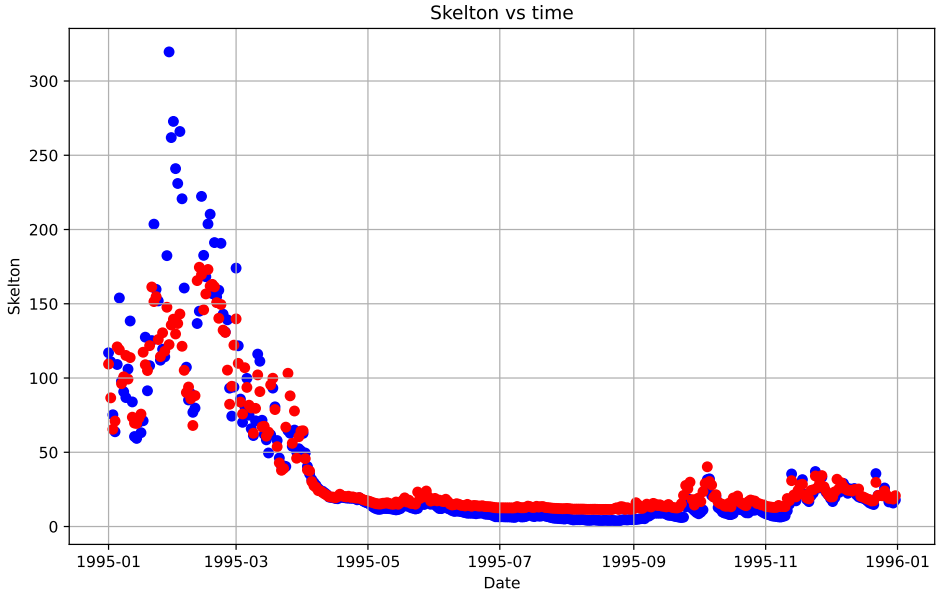


Figure 2.10.5: Predicted vs. Expected Output for Dynamic Batch Size

With large numbers of epochs this implementation looses its speed advantage and becomes the same as thne object-oriented implementation.

2.11 Momentum

Momentum is a tequnique used to speed up the learning process of the network. If the gradient of the error graph is heading in the same direction, the learning rate is increased, propelling the network to learn faster. This compounds the changes to remove large errors and speed up the learning process.

$$\text{Matrix } \omega_{nq}^{k} : \omega_{nq}^{\prime k} = \omega_{nq}^{k} + \eta \cdot \alpha_{n}^{k-1} \cdot \delta_{q}^{k} + \Delta\omega_{nq}^{k} \tag{2.11.1}$$

Where $\Delta\omega_{nq}^{k}$ is the momentum value.

back_prop_momentum.py

```
    class MLP:

        def backward_pass(self, expectedOutput:np.array, summedList:np.array, activatedList:np.array, inputData:np.array):
            '''Function to pass data through the network'''

            # Calculate the delta values for the network
            deltaList = self.delta_calculator(expectedOutput, activatedList, summedList)


            #  Set previous output as input values
            previousOutput = inputData

            # Iterate through the layers of the network
            for layer in range(len(self.nodeStructure)):

                # Update the weights and biases for the layer
                weightUpdate  = ((np.transpose(previousOutput, (0, 2, 1)) @ deltaList[layer]) * self.learningRate)
                biasUpdate = (deltaList[layer] * self.learningRate)

                # Update the previous output
                previousOutput = activatedList[layer]

                # Momentum update
                momentumValue = (self.weightList[layer] - self.previousWeightList[layer]) * self.momentumConstant

                # Update the weights and biases
                self.weightList[layer] += (weightUpdate.mean(axis=0, dtype=np.float64) + momentumValue)
                self.biasList[layer] += biasUpdate.mean (axis=0, dtype=np.float64)

            # Momentum update
```

```
33                self.previousWeightList = self.weightList
```

The code shows how the momentum value is implemented and included in the weight update. As each weight update is a result of the last, this compounds the changes and speeds up the learning process. However, at smaller changes, the momentum can cause the network to overshoot the minimum, and so the momentum constant must be carefully chosen.

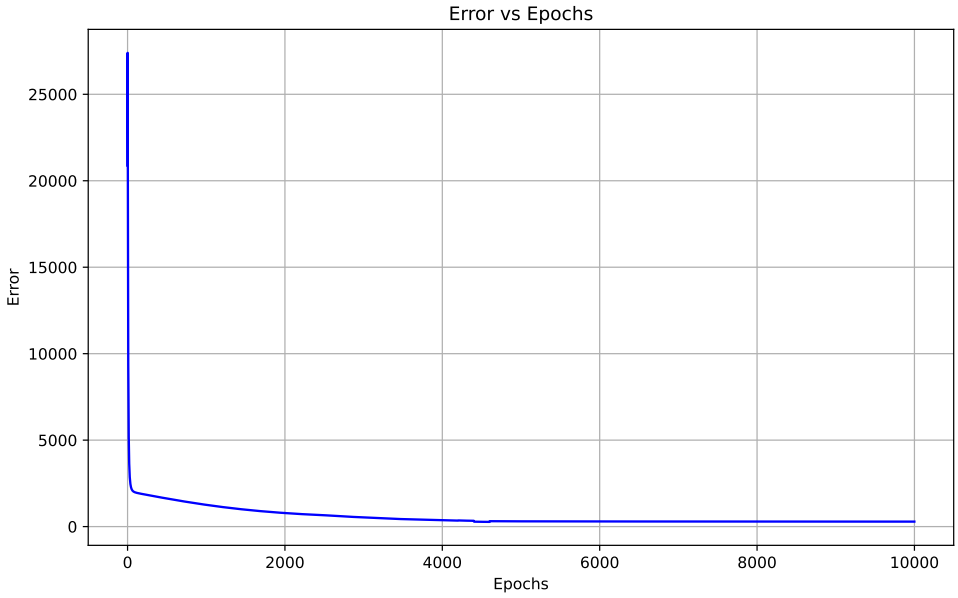


Figure 2.11.1: Error vs Epoch for Momentum

Time Taken (seconds)	Error (MSE)
407	287

Table 2.5: statistics for 10,000 epochs of Momentum

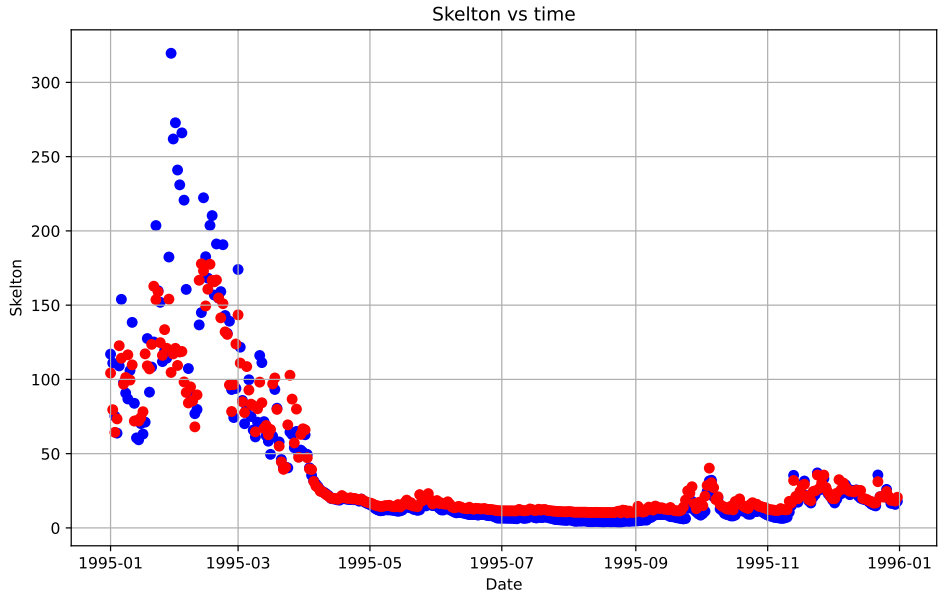


Figure 2.11.2: Predicted vs Expected for Momentum

This implementation tells us that Momentum allows us to reach lower errors faster, but then struggles to adapt after that point. To help adjust this, we can reduce the momentum constant afteer we reach a certain gradient, just as with the batch learning implementation. This can help us to fine-tune the network and reach a lower error. While assessing

this, I noticed my implementation of the gradient for lower batches was incorrect, as it use the whole gradient, not the local gradient. This rapidly increases batch size, which is not efficient or effective.

back_prop_momentum.py

```
    class mlp

        def train_network(self) -> pd.DataFrame:
            '''Function to train the neural network'''

            # Iterate through the training epochs
            for epoch in range(self.trainingEpochs):

                # Define the batch data
                batchData = np.array_split(self.trainingData, self.batchSize)

                # Define the mean error list
                meanErrorList = []

                # Iterate through the batches
                for batch in batchData:

                    # Split the data point into input and output, maintaining the 3D structure
                    inputData, expectedOutput = batch[:, :, 2:], batch[:, :, 1:2]

                    # Pass the data through the network
                    activatedList, summedList = self.forward_pass(inputData)

                    # Calculate the error of the network
                    meanErrorbatch = self.error_storing(expectedOutput, activatedList[-1])

                    # Reshape dimensions
                    meanErrorbatch = meanErrorbatch.squeeze(axis=1)

                    # Average the outputs and store in the list
                    meanErrorList.append(meanErrorbatch.mean())

                    # Backward pass through the network
                    self.backward_pass(expectedOutput, summedList, activatedList, inputData)


                # Convert errors into dataframe
                meanErrorDF = pd.DataFrame({'Error' : meanErrorList})

                # Calculate the mean error for the epoch
                meanError = meanErrorDF['Error'].mean()

                # Store the mean error for the epoch
                newError = pd.DataFrame({'Epoch': [epoch], 'Error': [meanError]})
                self.epochErrorDF = pd.concat([self.epochErrorDF, newError], ignore_index=True)


                # Only allow changes every 200 epochs to force the network to learn at certain points, and
    enforces at least 2 epochs
                if epoch % 200 == 0 and len(self.epochErrorDF) > 1:

                    # Define the last 200 points for an accurate gradient
                    recentErrors = self.epochErrorDF['Error'][-200:] if len(self.epochErrorDF) >= 200 else
    self.epochErrorDF['Error']

                    # Calculate the error gradient with respect to the batch size
                    errorGradient = np.gradient(recentErrors)

                    print(f'Error Gradient: {errorGradient[-1]}')

                    # Doesn't allow batch size to be greater than the training data
                    if self.batchSize == len(self.trainingData):
                        None

                    # Increase the batch size
                    elif abs(errorGradient[-1]) < 0.2:
                        self.batchSize = self.batchSize * 2
                        self.momentumConstant = self.momentumConstant * 0.6

                        # Forces batch size to be the same as the training data
                        if self.batchSize > len(self.trainingData):
                            self.batchSize = len(self.trainingData)


                # Print the error for the epoch
                if epoch % 100 == 0:
```

```
77                 print(f'Epoch: {epoch}, Error: {meanError}, Batch Size: {self.batchSize}')
```

This is my dynamic reduction in momentum constant, which allows for the network to learn faster and then fine-tune the network.

Time Taken (seconds)	Error (MSE)
400	287

Table 2.6: statistics for 10,000 epochs of Dynamic Momentum

This improvement is not very significant, as the only increase is it is 7 seconds faster, which is a 1.7% increase in speed. In the last 5000 epochs, the MSE changed by 5. There was also a low point just before at 276, which was skipped over. The take-away from this is that the smaller the error, the slower it learns.

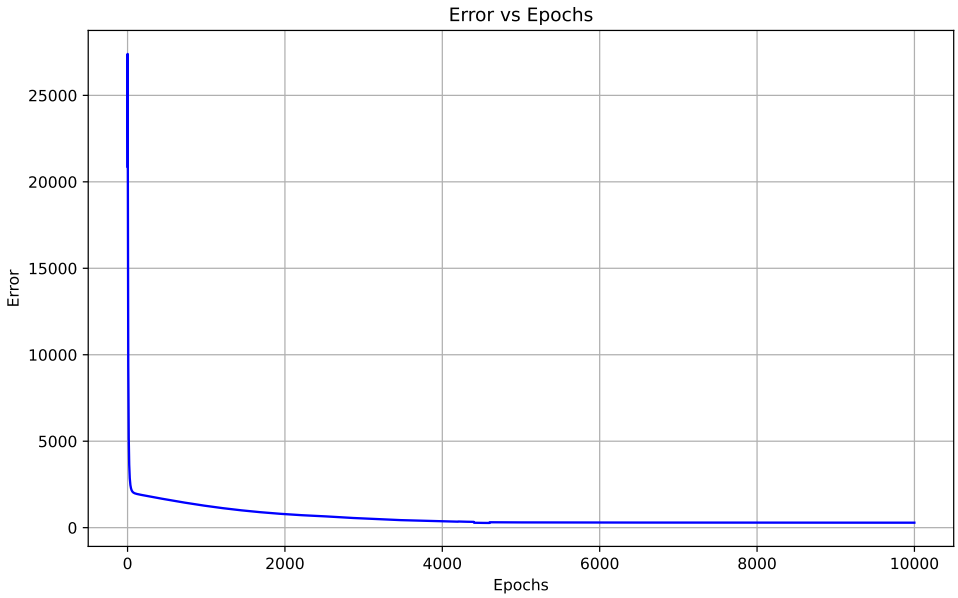


Figure 2.11.3: Error vs Epoch for Dynamic Momentum

2.12 Bold Driver

Bold driver is a tequnique that allows us to increase the learning rate when the gradient flattens out, allowing our network to still learn efficiently. This can be done with bounds and dynamic learning rate, which can be adjusted to the gradient of the error.

Bold drivers counter part is Anealing. This is where the learning rate is decreased as the gradient flatterns to help even out the network and stabalise it. This helps you reach a minimum point quicker and then not overshoot it, but it can also slow down the learning process. As my networks are already stable and are suffering from insifficient learning, I will not be implementing Anealing.

back_prop_bold_driver.py

```
2
3       class MLP:
4
5           def __init__(self, dataSet:pd.DataFrame, nodeStructure:list[tuple], learningRate:float,
    trainingEpochs:int, PredictedColumn:str):
6               '''Initialises the perceptron structure'''
7
8               # Create Bold Driver Learning Rate
9               self.boldDriverLearningRate = learningRate
10              self.boldDriverCap = 0.8
11              self.boldDriverIncrease = 1.2
12
13          def train_network(self) -> pd.DataFrame:
14              '''Function to train the neural network'''
15
16              # Only allow changes every 200 epochs to force the network to learn at certain points, and
    enforces at least 2 epochs
17              if epoch % 200 == 0 and len(self.epochErrorDF) > 1:
18
```

```
                # Define the last 200 points for an accurate gradient
                recentErrors = self.epochErrorDF['Error'][-200:] if len(self.epochErrorDF) >= 200 else self.
    epochErrorDF['Error']

                # Calculate the error gradient with respect to the batch size
                errorGradient = np.gradient(recentErrors)

                print(f'Error Gradient: {errorGradient[-1]}')

                # Doesn't allow batch size to be greater than the training data
                if self.batchSize == len(self.trainingData):
                    None

                # Increase the batch size
                elif abs(errorGradient[-1]) < 0.3:
                    self.batchSize = self.batchSize * 2
                    self.momentumConstant = self.momentumConstant * 0.6

                    # Forces batch size to be the same as the training data
                    if self.batchSize > len(self.trainingData):
                        self.batchSize = len(self.trainingData)

                # Update the learning rate
                if abs(errorGradient[-1]) < 0.07:
                    self.boldDriverLearningRate = self.boldDriverLearningRate * self.boldDriverIncrease

                    # Caps the learning rate
                    if self.boldDriverLearningRate > (1+self.boldDriverCap)*self.learningRate:
                        self.boldDriverLearningRate = (1+self.boldDriverCap)*self.learningRat
```

This code shows how the bold driver learning rate is implemented and included in the weight update.

Time Taken (seconds)	Error (MSE)
434	282

Table 2.7: statistics for 10,000 epochs of Bold Driver

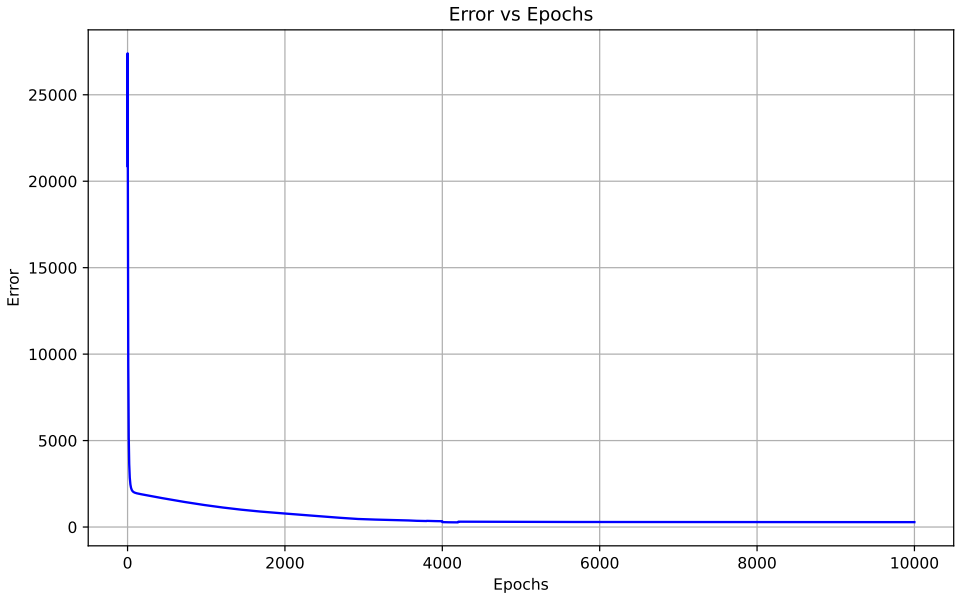


Figure 2.12.1: Error vs Epoch for Bold Driver

The bold driver implementation does reduce the error, but not significantly. It is possible our bold driver constants are not extreme enough, and so the learning rate is not changing enough.

back_prop_bold_driver.py

```
    self.boldDriverCap = 20
```

By increasing the cap, we can allow the network to learn more at these lower gradients.

Time Taken (seconds)	Error (MSE)
371	238

Table 2.8: statistics for 10,000 epochs of Bold Driver

This is the best result so far, with a 14% increase in speed and a 16% decrease in error. This is a significant improvement and shows that the bold driver implementation is much more efficient.

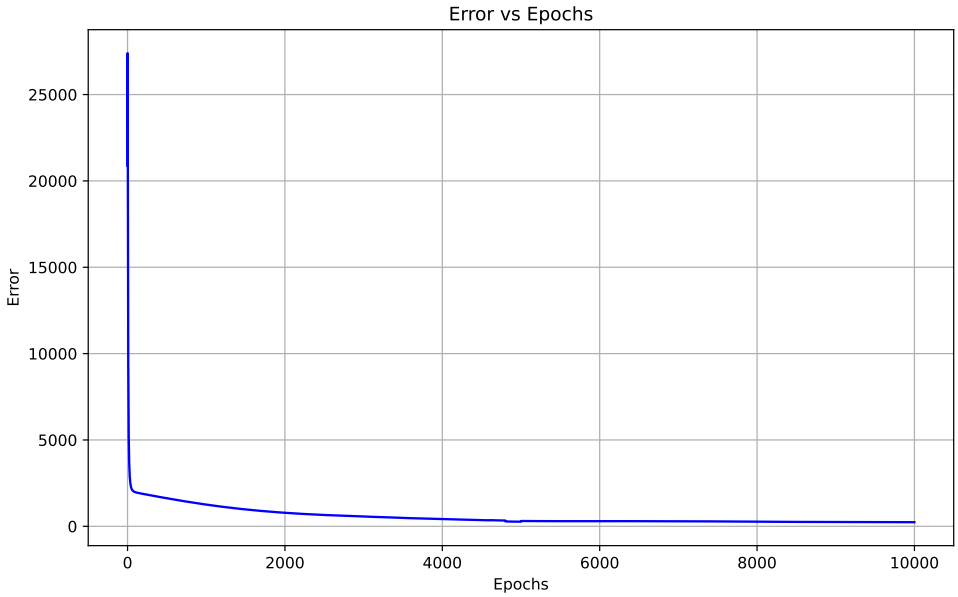


Figure 2.12.2: Error vs Epoch for Bold Driver (2^{nd} Implementation)

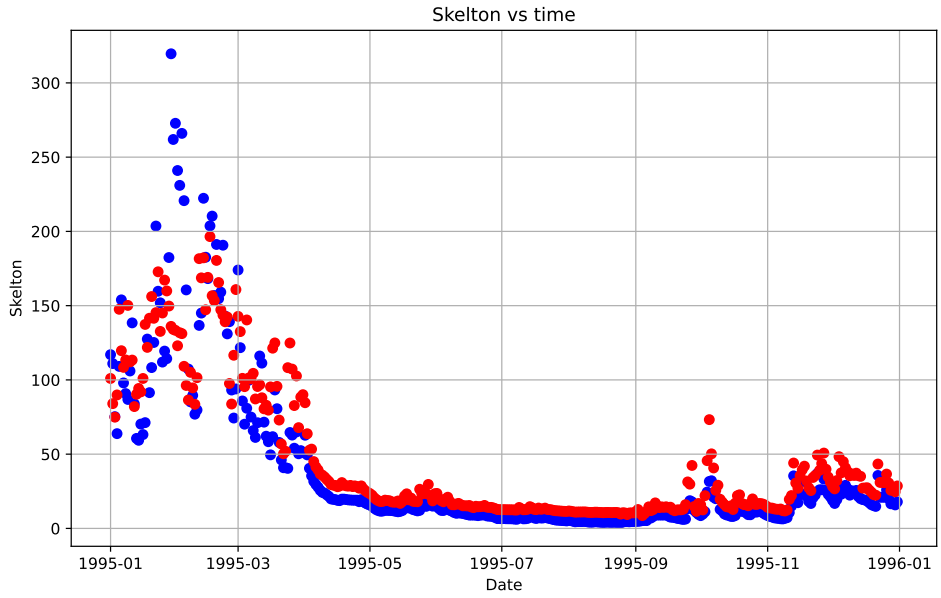


Figure 2.12.3: Predicted vs Expected for Bold Driver (2^{nd} Implementation)

2.13 Weight Decay

Weight decay incetiveses the network to produce smaller weights, which can help to prevent overfitting. It does this by adding a penalty term to error function, which is proportional to the size of the weights.

$$\text{Matrix } \omega_{nq}^{k} : \omega_{nq}'^{k} = \omega_{nq}^{k} + \eta \cdot \alpha_{n}^{k-1} \cdot \delta_{q}^{k} + \Delta\omega_{nq}^{k} - \rho\omega_{nq}^{k} \tag{2.13.1}$$

Where $\rho\omega_{nq}^{k}$ is the weight decay term.

This term forces weights to be much smaller, and so the network is less likely to overfit.

back_prop_weight_decay.py

```
    class MLP:
        def __init__(self, dataSet:pd.DataFrame, nodeStructure:list[tuple], learningRate:float, trainingEpochs:int, PredictedColumn:str):
            '''Initialises the perceptron structure'''

            # Create weight decay constant
            self.weightDecay = 0.0001

        def backward_pass(self, expectedOutput:np.array, summedList:np.array, activatedList:np.array, inputData:np.array):
            '''Function to pass data through the network'''

            # Calculate the delta values for the network
            deltaList = self.delta_calculator(expectedOutput, activatedList, summedList)


            #  Set previous output as input values
            previousOutput = inputData

            # Iterate through the layers of the network
            for layer in range(len(self.nodeStructure)):

                # Update the weights and biases for the layer
                weightUpdate  = ((np.transpose(previousOutput, (0, 2, 1)) @ deltaList[layer]) * self.boldDriverLearningRate)
                biasUpdate = (deltaList[layer] * self.boldDriverLearningRate)

                # Update the previous output
                previousOutput = activatedList[layer]

                # Momentum update
                momentumValue = (self.weightList[layer] - self.previousWeightList[layer]) * self.momentumConstant

                # Weight Decay update
                weightDecay = self.weightList[layer] * self.weightDecay

                # Update the weights and biases
                self.weightList[layer] += (weightUpdate.mean(axis=0, dtype=np.float64) + momentumValue - weightDecay)
                self.biasList[layer] += biasUpdate.mean (axis=0, dtype=np.float64)

            # Momentum update
            self.previousWeightList = self.weightList
```

This code shows how the weight decay term is implemented and included in the weight update.

Time Taken (seconds)	Error (MSE)
405	1313

Table 2.9: statistics for 10,000 epochs of Weight Decay

This is the worst result so far, with a 361% increase in error. This may be due to the weight decay term being too large, and so the network is not learning effectively.

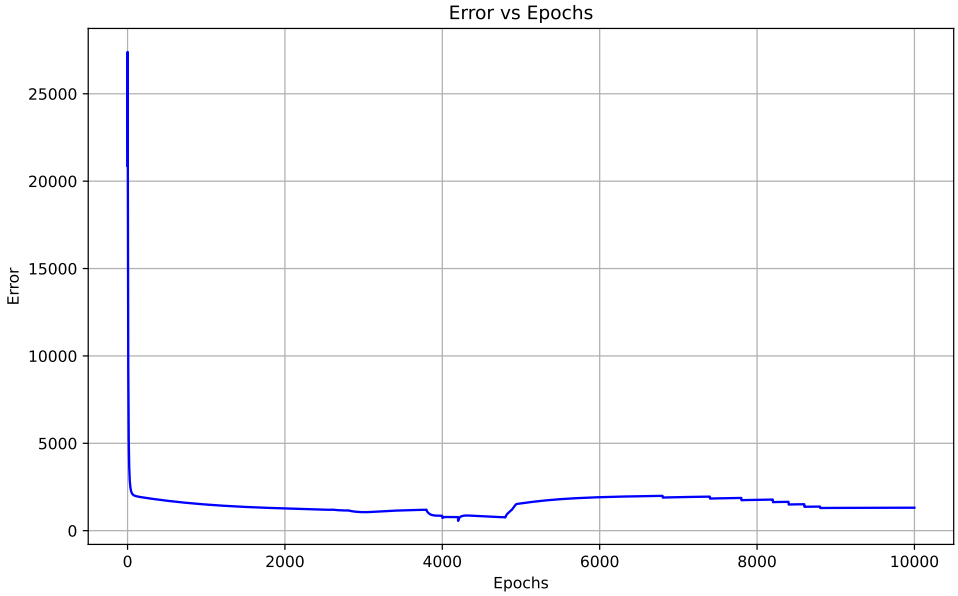


Figure 2.13.1: Error vs Epoch for Weight Decay

```
back_prop_weight_decay.py

    self.weightDecay = 0.00001
```

By reducing the weight decay term, we can allow the network to learn more effectively.

Time Taken (seconds)	Error (MSE)
375	303

Table 2.10: statistics for 10,000 epochs of Weight Decay (2^{nd} Implementation)

These results do not match the results without the weight decay as they only got worse. I do not beleive that the network suffered from overfitting in previous implementations, and so the weight decay term was not needed. For this reason, I will not be using weight decay in my final implementation.

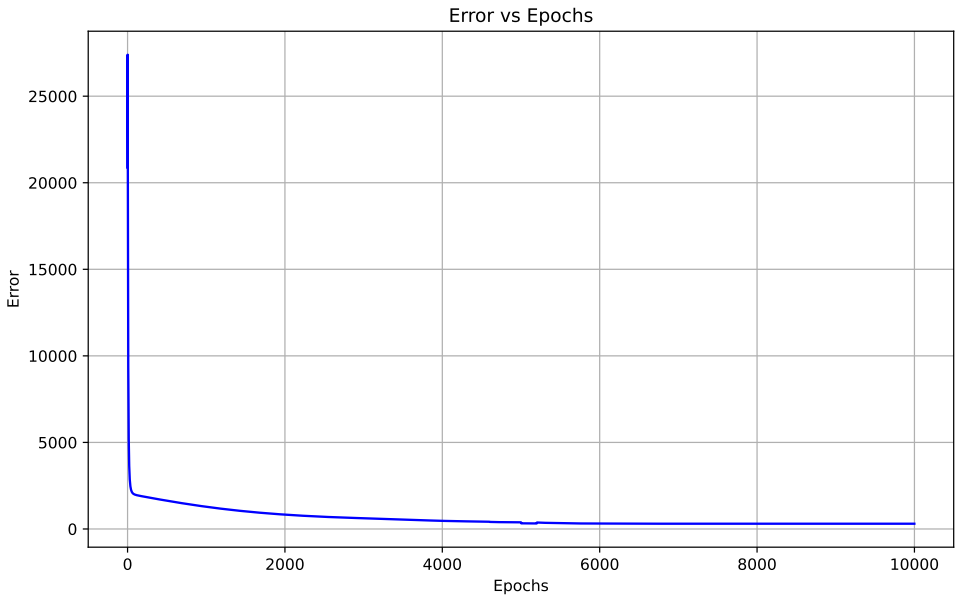


Figure 2.13.2: Error vs Epoch for Weight Decay (2^{nd} Implementation)

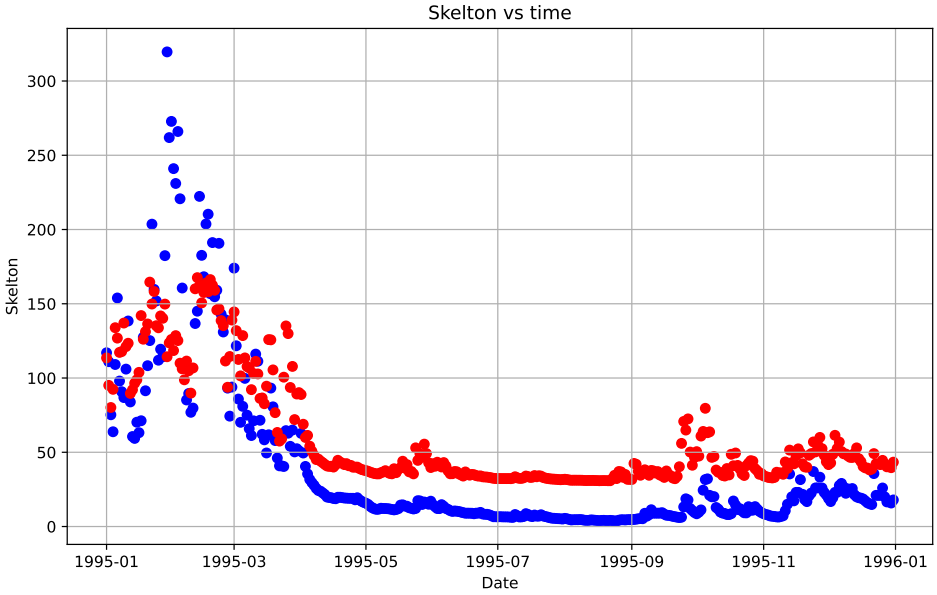


Figure 2.13.3: Predicted vs Expected for Weight Decay (2^{nd} Implementation)

2.14 Line Searching

Learn searching is about using the gradient to adjust the learning rate of network to be more efficient and avoid overshooting the minimum. We can define our output as a function of the learning rate and direction of the gradient.

$$O_k = O_{k-1} + \sigma d_k \tag{2.14.1}$$

Where O_k is the output, O_{k-1} is the previous output, σ is the learning rate and d_k is the direction of the gradient. 1em As we are optimising in terms of the learning rate, we can frame this as a minimisation problem of function ϕ

$$\phi(\sigma) = f(O_{k-1} + \sigma d_k) \tag{2.14.2}$$

Where ϕ is the function we are trying to minimise, f is the function we are trying to optimise. This means that we can use the gradient of ϕ to find the optimal learning rate as we use it to find the minimum of the function.

```
back_prop_line_search.py

    class MLP:

        def line_search(self, expectedOutput:np.array, activatedList:np.array, summedList:np.array, inputData
    :np.array):
            '''Function to perform a line search and optimise the learning rate'''

            # Define the next two steps
            learningRateAlpha = self.learningRate
            learningRateBeta = self.learningRate * 50

            # Define the initial error and delta list
            deltaList, structureCost = self.delta_calculator(expectedOutput, activatedList, summedList)

            # Calculate the weights for the first step

            # Calculate the forward pass for the first step
            activatedListAlpha, summedListAlpha = self.forward_pass(inputData, self.weightList, self.biasList
    )

            # Calculate the error for the first step
            structureCostAlpha = self.error_storing(expectedOutput, activatedListAlpha[-1])

            # Calculate the weights for the second step

            # Calculate the forward pass for the second step
            activatedListBeta, summedListBeta = self.forward_pass(inputData, self.weightList, self.biasList)

            # Calculate the error for the second step
            structureCostBeta = self.error_storing(expectedOutput, activatedListBeta[-1])

            # Use a quadratic model to approximate the minimum learning rate
```

```
            X = np.array([
                [0**2, 0, 1],
                [learningRateAlpha**2, learningRateAlpha, 1],
                [learningRateBeta**2, learningRateBeta, 1]
            ])


            Y = np.array([structureCost.mean(), structureCostAlpha.mean(), structureCostBeta.mean()])

            # Solve for quadratic coefficients [a, b, c]
            a, b, c = np.linalg.solve(X, Y)

            # Compute the minimum learning rate
            stepSize = -b / (2 * a)

            # Calculate the backwards pass for the chose step size
            newWeightList, newBiasList = self.backward_pass(activatedList, inputData, stepSize, deltaList)

            self.weightList = newWeightList
            self.biasList = newBiasList

            # Momentum update
            self.previousWeightList = newWeightList

            def backward_pass(self, activatedList:np.array, inputData:np.array, lineSearchConstant:float, deltaList:np.array):
                '''Function to pass data through the network'''

                # Create a list for the temporary weights and biases
                temporaryWeightList = []
                temporaryBiasList = []

                #  Set previous output as input values
                previousOutput = inputData

                # Define the step size for the line search
                stepSize = self.boldDriverLearningRate * lineSearchConstant

                # Iterate through the layers of the network
                for layer in range(len(self.nodeStructure)):

                    # Update the weights and biases for the layer
                    weightUpdate  = ((np.transpose(previousOutput, (0, 2, 1)) @ deltaList[layer]) * stepSize)
                    biasUpdate = (deltaList[layer] * stepSize)

                    # Update the previous output
                    previousOutput = activatedList[layer]

                    # Momentum update
                    momentumValue = (self.weightList[layer] - self.previousWeightList[layer]) * self.momentumConstant

                    # Update the weights and biases
                    temporaryWeightList.append(self.weightList[layer] + (weightUpdate.mean(axis=0, dtype=np.float64) + momentumValue))
                    temporaryBiasList.append(self.biasList[layer] + biasUpdate.mean (axis=0, dtype=np.float64))

                return temporaryWeightList, temporaryBiasList
```

My implementation of ϕ is to use the current cost, along with 2 potential costs to approximate the minimum of a quadratic function. This minumum will be used as the step size for the line search and then used to update our weights. This is a costly operation, as it requires 3 backward passes and 3 forward passes per input. With the batch learning, I hope to mitigate the cost of this operation.

Time Taken (seconds)	Error (MSE)
805	170

Table 2.11: statistics for 10,000 epochs of Line Searching

This is the best result so far, but with an expected increase in time taken.

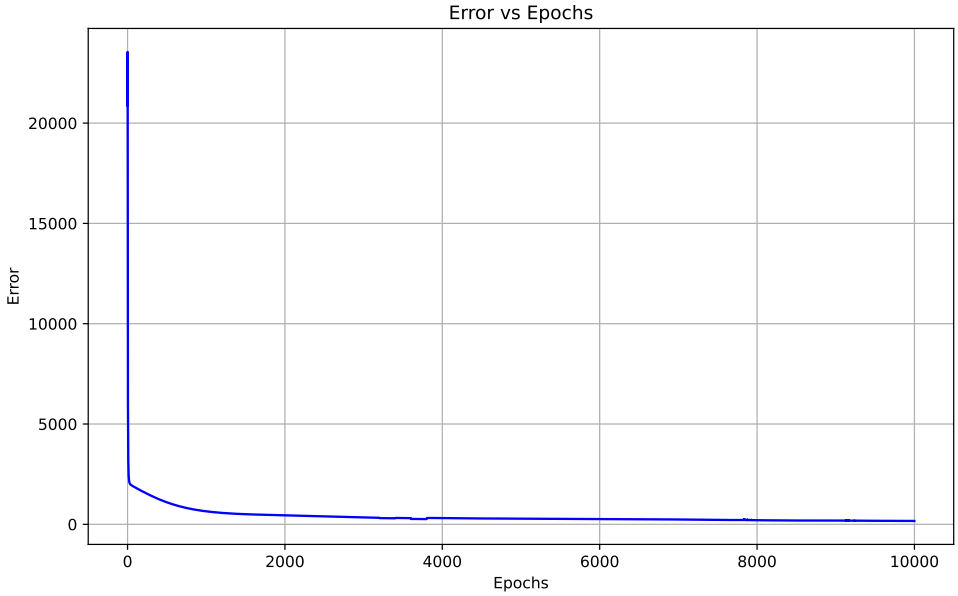


Figure 2.14.1: Error vs Epoch for Line Searching

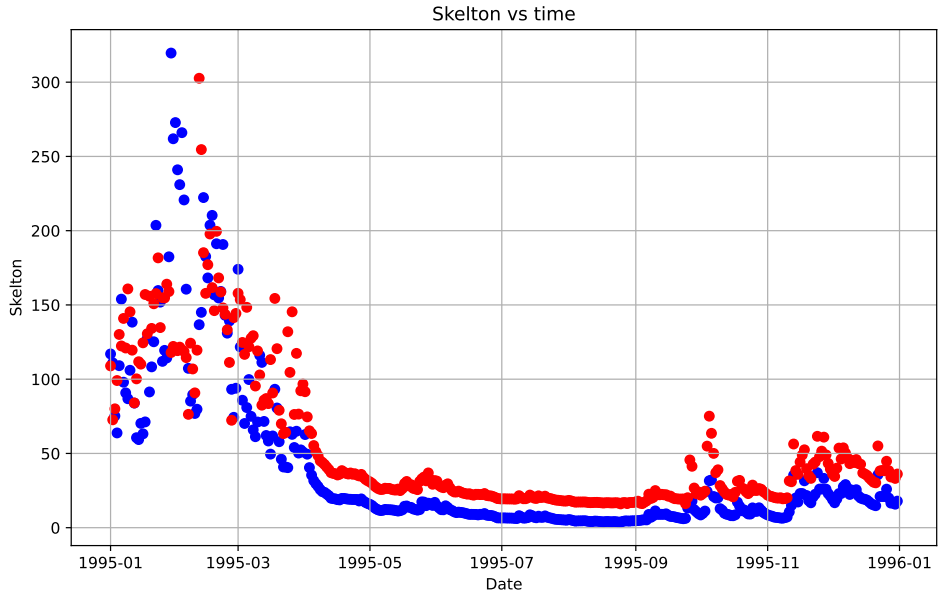


Figure 2.14.2: Predicted vs Expected for Line Searching

While the plots may be further away, the accuracy of the network is much higher, and the predictions at the first section of the year are much better. This is a significant improvement and shows that the line search implementation is much more efficient. However it is clear that their is an issue, as all the datapoints have been shifted upwards. Once rectified, this will be the best implementation of the network.

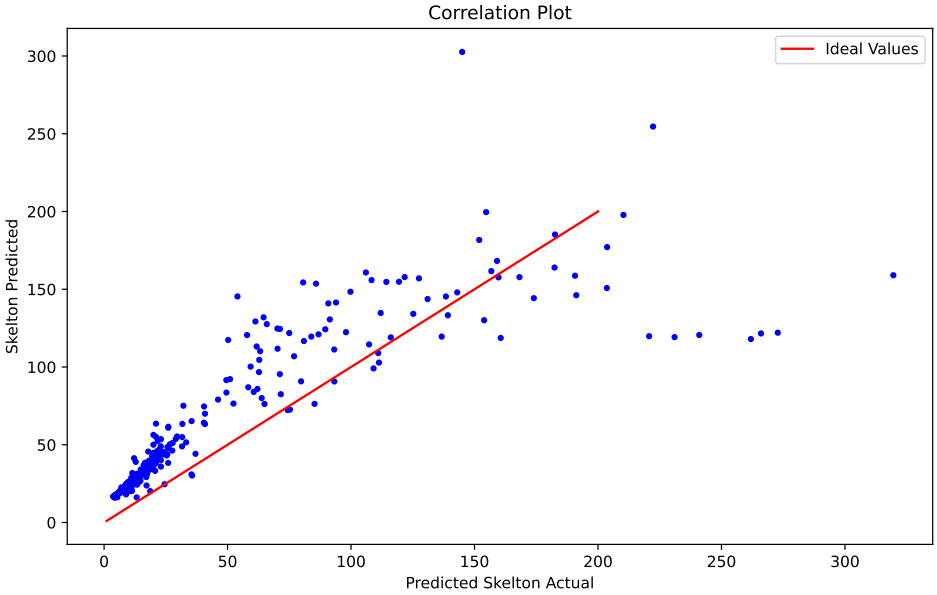


Figure 2.14.3: Skew for Line Searching

After analysis, the model was overtraining as the batch size decreases. By capping the batch size to 32, the model was able to learn more effectively and not overfit. This is likely due to the line search being more effective with smaller batch sizes, and so the network was learning too much.

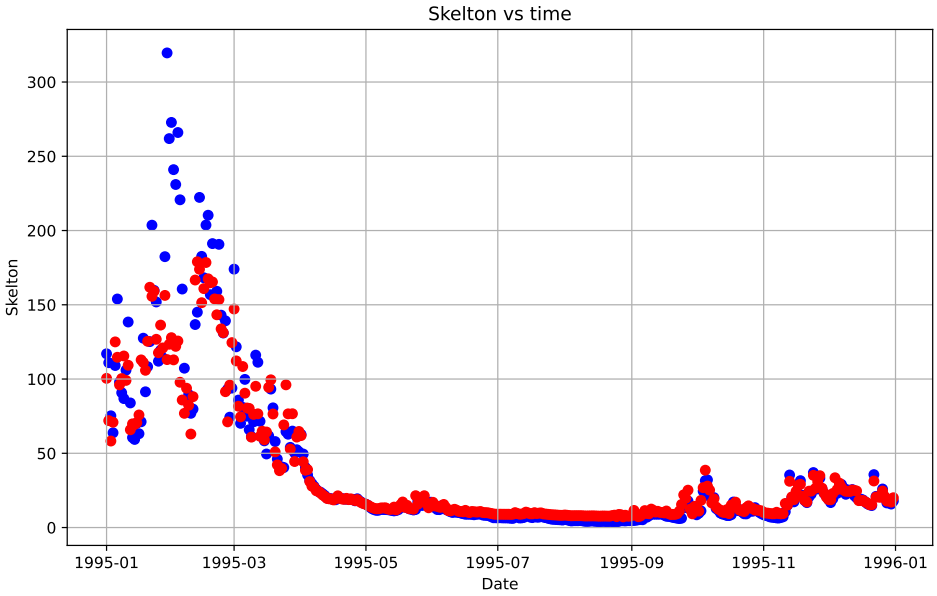


Figure 2.14.4: Predicted vs Expected for Line Searching (2^{nd} Implementation)

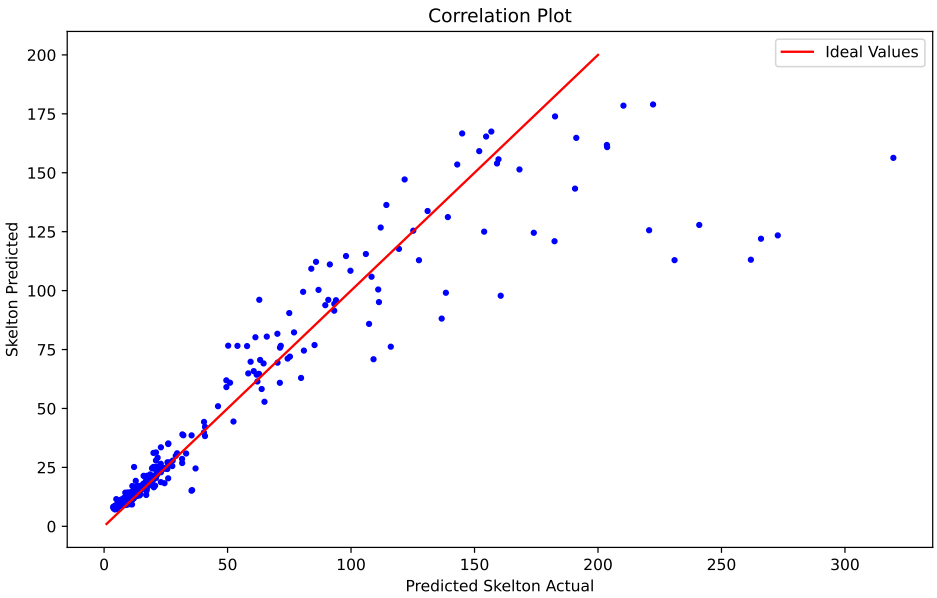


Figure 2.14.5: Correlation Plot for Line Searching (2^{nd} Implementation)

Chapter 3

Model training and testing

Most model training and testing occured in the previous chapter, but we will now evaluate the best model and see how well it predicts the data.

The best model was the line search model, with a batch size of 32. This model was able to learn the most effectively and produce the best results in the quickest amount of time. As all model simulations were done with the same random seed and with 10,000 epochs, the results may not be as good as they can be.

We will take the model we chose and run it for 50,000 epochs and evaulate the error of the testing data to see how well the model predicts.

Time Taken (seconds)	Error (MSE)
919	195

Table 3.1: statistics for 50,000 epochs of Line Searching

When the model was trained for 50,000 epochs, the error was reduced to 195 on the evaluation data, which is a significant improvement. It ended on an MSE of 141 on the training data, which is a significant improvement on previous models. However, when run for 100,000 epochs, the model became overfit and the MSE increased to 1000 on the evaluation data.

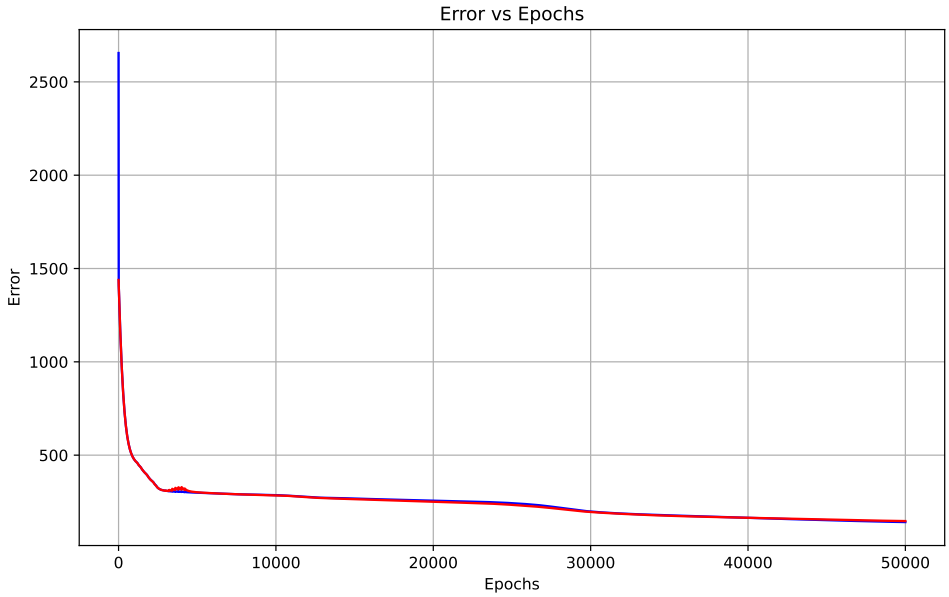


Figure 3.0.1: Error vs Epoch for Line Searching (50,000 epochs)

The red line represents the error of the evaluation data, and the blue line represents the error of the training data.

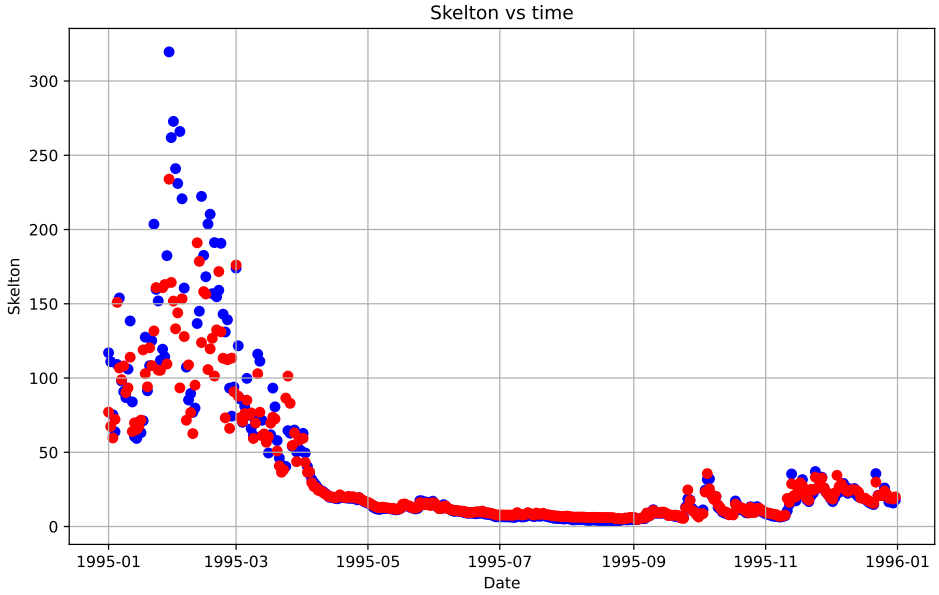


Figure 3.0.2: Predicted vs Expected for Line Searching (50,000 epochs)

The model matches very well to the expected output, and the correlation is very high for the steady data. When the data becomes more sporadic, the model struggles with this

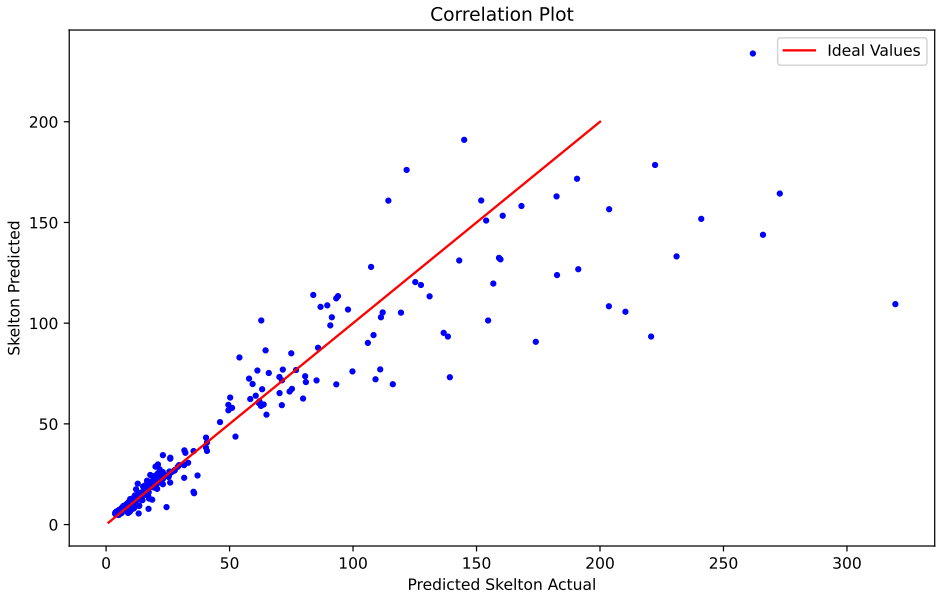


Figure 3.0.3: Correlation Plot for Line Searching (50,000 epochs)

Chapter 4

Evaluation

4.1 Model Evaluation

The provided model was able to predict the error with a mean squared error of 195, and a time of 919 seconds The development of the model was costly, and the time taken to train the model was significant.

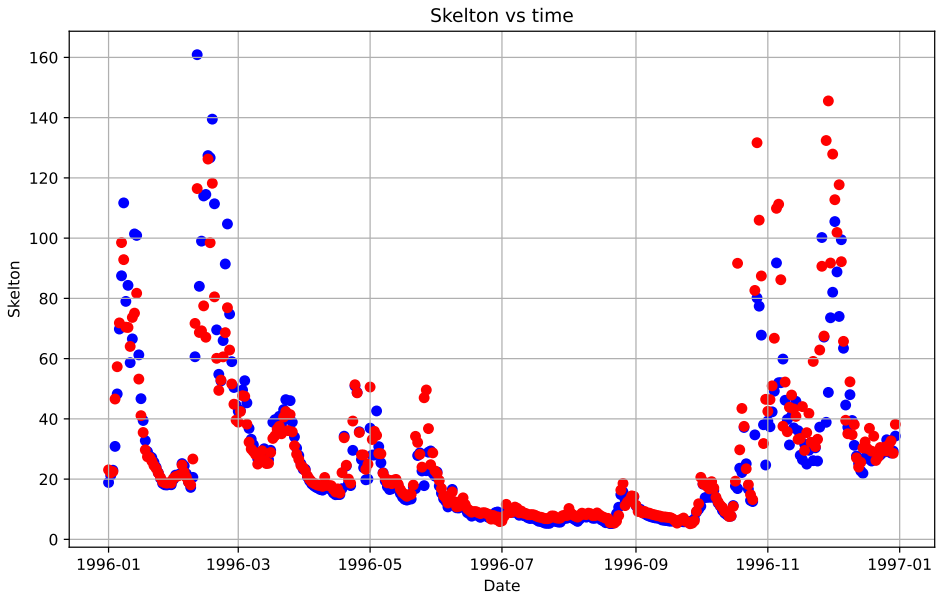


Figure 4.1.1: Predicted vs Expected for Line Searching (50,000 epochs)

The predicted evaluation data is shown in red, and the expected evaluation data is shown in blue. It is obvious that the model is not able to predict the sporadic data as well as the steady data, however the model is acceptable. The MSE of 195 gives an average difference of 14, which clearly takes more affect at the sporadic data.

4.2 Model Comparison

We can compare the model with a basic linear regression algorithm to see how well the model performs. This will allow us to see how well the model performs compared to a simple model.

Time Taken (seconds)	Error (MSE)
0.17	119

Table 4.1: statistics for Linear Regression

The linear regression model beats the neural network model in both metrics. This is likely because there is a largely linear relationship between the inputs and output, and so a linear regression model is the best model for this data. All 7 inputs are highly correlated and it intuitively follows that they have a high linear correlation.

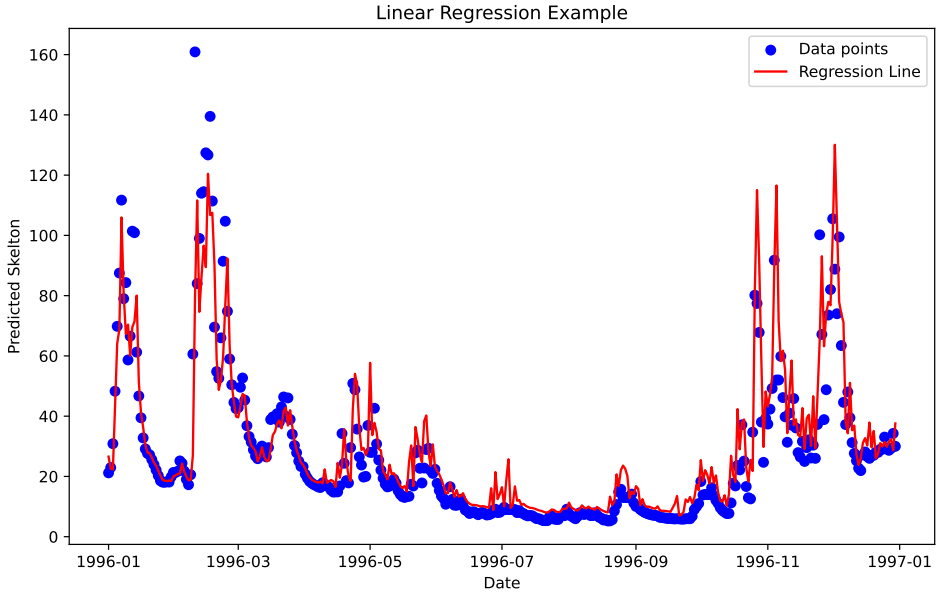


Figure 4.2.1: Expected vs Predicted for Linear Regression

The linear regression model is able to predict the data very well, with a MSE of 119. This means that the average difference is 11 between the expected and predicted data. This is a significant improvement on the neural network model, but mainly interms of time and effort.

A differnce of 11 and 14 is not a significant difference, but the time taken to train the model is.

4.3 Conclusions

The neural network model is able to predict the data well. It used 50% of the data and trained for 50,000 epochs. With the chosen model this was a 195 MSE on the final evaluation data. I would conclude that this neural network is an effective way of predicting the data, but it is not the best way.

The linear regression model is able to predict the data better, with a 119 MSE on the final evaluation data. The linear regression model takes only 0.17 seconds to train, which is significantly faster than the neural network model, around 6000 times faster. While the training time is faster, so is the amount of time spent developing the model.

In conclusion, the Linear Regression model is the best model for this data, as it is able to predict the data better and faster than the neural network model.

4.4 Potential Improvements

If I was to improve my network, I would try to work with different structures and optimise there. I only used 1 Cost funciton and 1 Activation function, and so I could try different ones to see if they are more effective. I could also try to vary the node structure and see if that is more effective with more layers or nodes. I could also try to implement a seperate gradient calculation, such as conjufate gradient searching.

Overall I am please with my results and for this purpose, The model is effective and efficient.

4.5 Extra Plots

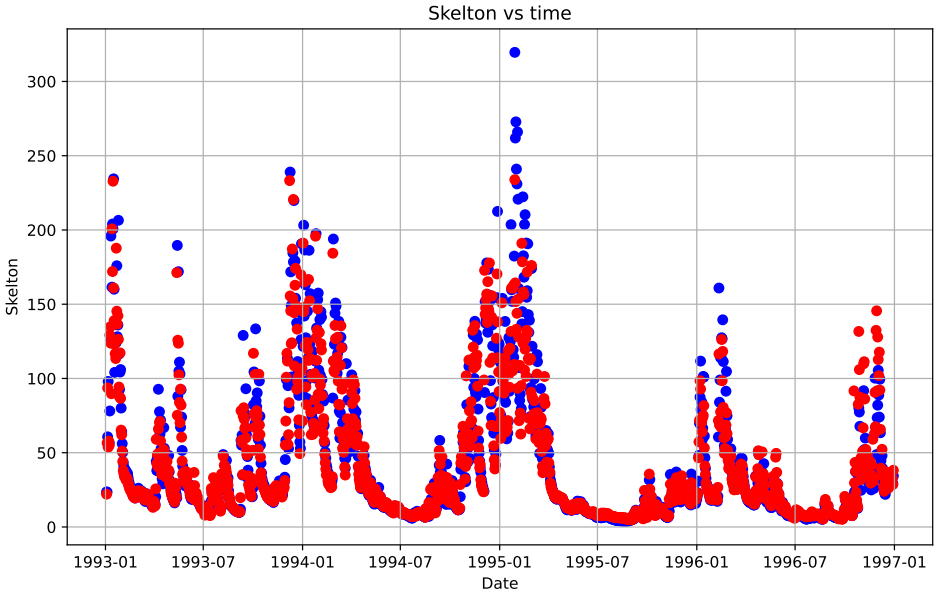


Figure 4.5.1: Predicted vs Expected for Line Searching (50,000 epochs)

Predicted vs Expected for the whole given dataset.

Figure 4.5.2: Correlation for Line Searching (50,000 epochs)

Correlation between predicted and expected data for the whole given dataset.